

ATS Assessment - Phase 3

Hedera Hashgraph SaaS

HALBORN

Summary

100% OF ALL REPORTED FINDINGS HAVE BEEN ADDRESSED

ABOUT THIS REPORT

ASSESSMENT TYPE

Assurance Assessment

SOURCE CODE

 [asset-tokenization-studio](#)

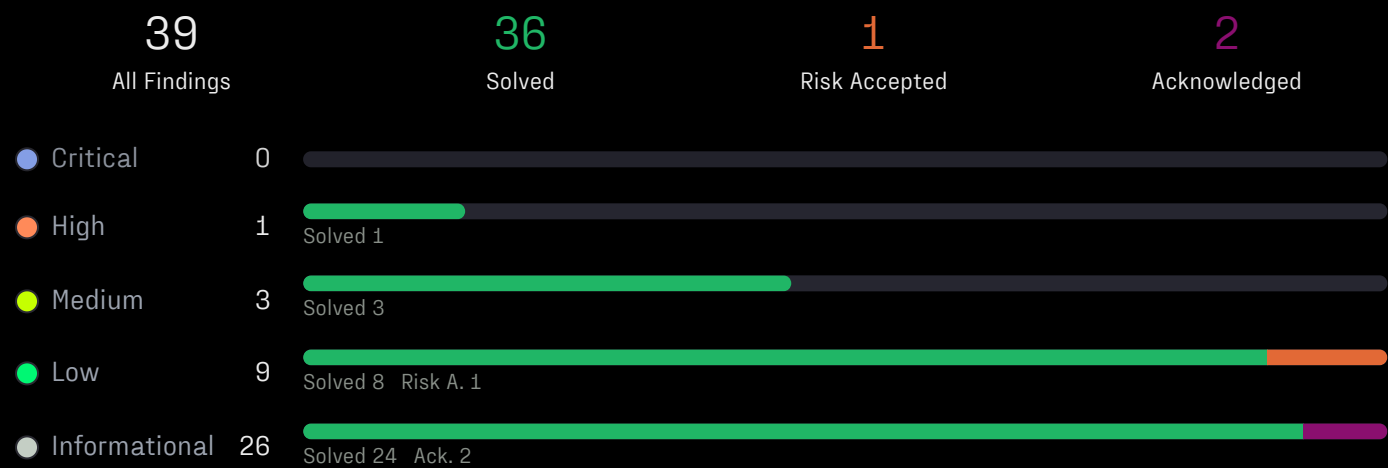
DATE OF ENGAGEMENT

May 19, 2026 - Jun 4, 2026

ASSESSED COMMIT ID

 c1b3835

SEVERITY BREAKDOWN



Introduction

Hashgraph engaged **Halborn** to perform a security assessment of their Asset Tokenization Studio Phase 3 (ATS) smart contracts from **May 19th, 2026 to June 3rd, 2026**. The assessment scope was limited to the smart contracts provided to Halborn, commit hashes and additional details are available in the Scope section of this report.

The **Asset Tokenization Studio (ATS)** is an institutional-grade security token platform built on the ERC-3643 (T-REX) standard, designed for the issuance, management, and lifecycle administration of tokenized financial instruments including equities and bonds. The platform implements a Diamond Proxy architecture with a Business Logic Resolver for modular upgradeability, enabling tokens to be composed from interchangeable facets covering ERC-1400/ERC-1410 partitioned transfers, ERC-20 compatibility, compliance enforcement, and identity management.

Core financial features include bond coupon calculations with KPI-Linked and Sustainability Performance Target (SPT) interest rate models, Automatic Balance Adjustment Factor (ABAF) for corporate actions such as stock splits and reverse splits, a clearing and settlement module for atomic delivery-versus-payment flows, a hold management system (ERC-1996 inspired) for escrowed transfers, token locking with expiration-based release, snapshot-based historical balance

queries for record dates and dividend entitlements, and an ERC-20 Votes extension for on-chain governance. The platform enforces compliance through an integrated identity registry with KYC status tracking, revocation registry support, external control lists (whitelist/blacklist), and a pluggable compliance module architecture, while providing administrative controls via role-based access control, external pause management, and wallet recovery mechanisms. This Phase 3 assessment focused on the changes introduced since the Phase 2 review, including the storage-layout reorganization, the interest-rate and corporate-action logic, the clearing, hold, and scheduled-task subsystems, and the recovery, compliance, and access-control paths.

Assessment Summary

Halborn was allocated 12 business days for this engagement and assigned 1 full-time security engineer to conduct a comprehensive review of the smart contracts within scope. The engineer is an expert in blockchain and smart contract security, with advanced skills in penetration testing and smart contract exploitation, as well as extensive knowledge of multiple blockchain protocols and the ERC-3643 standard.

The objectives of this assessment were to:

- Identify potential security vulnerabilities within the smart contracts.
- Verify that the smart contract functionality operates as intended.
- Assess the correctness of the financial logic, including bond coupon calculations, ABAF adjustments, and clearing/settlement flows.

In summary, **Halborn** identified several areas for improvement to reduce the likelihood and impact of security risks, which were mostly addressed by the **Hashgraph** team. The primary recommendations were:

- **Restrict the batch partial freeze and unfreeze entrypoints to accounts holding a freeze role, matching the single-holder freeze functions, so that arbitrary callers cannot alter holder balances.**
- **Guard every facet initializer with access control so that only the deploying factory or a configured administrator can initialize an uninitialized facet.**
- **Revoke a lost wallet's privileged roles as part of the recovery flow, or reject recovery for a wallet that still holds roles, so a compromised key cannot retain privileged authority after recovery completes.**
- **Write the delegator's unadjusted balance into the voting checkpoint so the balance adjustment factor is not applied twice and governance tallies remain accurate.**
- **Enforce the pause state on the self-directed transfer path and apply the unpaused guard to the value-moving transfer and redeem entrypoints, so an**

emergency pause cannot be bypassed.

Risk Methodology

Every vulnerability and issue observed by Halborn is ranked based on **two sets of Metrics** and a **Severity Coefficient**. This system is inspired by the industry standard Common Vulnerability Scoring System.

The two **Metric sets** are: **Exploitability** and **Impact**. **Exploitability** captures the ease and technical means by which vulnerabilities can be exploited and **Impact** describes the consequences of a successful exploit.

The **Severity Coefficients** is designed to further refine the accuracy of the ranking with two factors: **Reversibility** and **Scope**. These capture the impact of the vulnerability on the environment as well as the number of users and smart contracts affected.

The final score is a value between 0-10 rounded up to 1 decimal place and 10 corresponding to the highest security risk. This provides an objective and accurate rating of the severity of security vulnerabilities in smart contracts.

The system is designed to assist in identifying and prioritizing vulnerabilities based on their level of risk to address the most critical issues in a timely manner.

3.1 EXPLOITABILITY

ATTACK ORIGIN (AO):

Captures whether the attack requires compromising a specific account.

ATTACK COST (AC):

Captures the cost of exploiting the vulnerability incurred by the attacker relative to sending a single transaction on the relevant blockchain. Includes but is not limited to financial and computational cost.

ATTACK COMPLEXITY (AX):

Describes the conditions beyond the attacker's control that must exist in order to exploit the vulnerability. Includes but is not limited to macro situation, available third-party liquidity and regulatory challenges.

METRICS:

EXPLOITABILITY METRIC (m_e)	METRIC VALUE	NUMERICAL VALUE
Attack Origin (AO)	Arbitrary (AO:A)	1
	Specific (AO:S)	0.2
Attack Cost (AC)	Low (AC:L)	1
	Medium (AC:M)	0.67
	High (AC:H)	0.33
Attack Complexity (AX)	Low (AX:L)	1
	Medium (AX:M)	0.67
	High (AX:H)	0.33

Exploitability E is calculated using the following formula:

$$E = \prod m_e$$

3.2 IMPACT

CONFIDENTIALITY (C):

Measures the impact to the confidentiality of the information resources managed by the contract due to a successfully exploited vulnerability. Confidentiality refers to limiting access to authorized users only.

INTEGRITY (I):

Measures the impact to integrity of a successfully exploited vulnerability. Integrity refers to the trustworthiness and veracity of data stored and/or processed on-chain. Integrity impact directly affecting Deposit or Yield records is excluded.

AVAILABILITY (A):

Measures the impact to the availability of the impacted component resulting from a successfully exploited vulnerability. This metric refers to smart contract features and functionality, not state. Availability impact directly affecting Deposit or Yield is excluded.

DEPOSIT (D):

Measures the impact to the deposits made to the contract by either users or owners.

YIELD (Y):

Measures the impact to the yield generated by the contract for either users or owners.

METRICS:

IMPACT METRIC (m_I)	METRIC VALUE	NUMERICAL VALUE
Confidentiality (C)	None (C:N)	0
	Low (C:L)	0.25
	Medium (C:M)	0.5
	High (C:H)	0.75
	Critical (C:C)	1
Integrity (I)	None (I:N)	0
	Low (I:L)	0.25
	Medium (I:M)	0.5
	High (I:H)	0.75
	Critical (I:C)	1
Availability (A)	None (A:N)	0
	Low (A:L)	0.25
	Medium (A:M)	0.5
	High (A:H)	0.75
	Critical (A:C)	1
Deposit (D)	None (D:N)	0
	Low (D:L)	0.25
	Medium (D:M)	0.5
	High (D:H)	0.75
	Critical (D:C)	1
Yield (Y)	None (Y:N)	0
	Low (Y:L)	0.25
	Medium (Y:M)	0.5
	High (Y:H)	0.75
	Critical (Y:C)	1

Impact I is calculated using the following formula:

$$I = \text{max}(m_I) + \frac{\sum m_I - \text{max}(m_I)}{4}$$

3.3 SEVERITY COEFFICIENT

REVERSIBILITY (R):

Describes the share of the exploited vulnerability effects that can be reversed. For upgradeable contracts, assume the contract private key is available.

SCOPE (S):

Captures whether a vulnerability in one vulnerable contract impacts resources in other contracts.

METRICS:

SEVERITY COEFFICIENT (C)	COEFFICIENT VALUE	NUMERICAL VALUE
Reversibility (r)	None (R:N)	1
	Partial (R:P)	0.5
	Full (R:F)	0.25
Scope (s)	Changed (S:C)	1.25
	Unchanged (S:U)	1

Severity Coefficient *C* is obtained by the following product:

$$C = rs$$

The Vulnerability Severity Score *S* is obtained by:

$$S = min(10, EIC * 10)$$

The score is rounded up to 1 decimal places.

Critical	High	Medium	Low	Informational
9 - 10	7 - 8.9	4.5 - 6.9	2 - 4.4	0 - 1.9

Scope

REPOSITORY
(a) Repository:  <code>asset-tokenization-studio</code>
(b) Assessed Commit ID: <code>♢ c1b38353bb72847ceb016a80a446b22084447847</code>
(c) Items in scope:

- ✓  contracts/contracts 524 files
 - ✓  constants 6 files
 - <> dispatchTypes.sol Solidity
 - <> eip1066.sol Solidity
 - <> eip712.sol Solidity
 - <> regulation.sol Solidity
 - <> roles.sol Solidity
 - <> values.sol Solidity
 - ✓  domain 54 files
 - ✓  asset 30 files
 - ✓  amortization 2 files
 - <> AmortizationStorageWrapper.sol Solidity
 - <> IAmortizationStorageWrapper.sol Solidity
 - ✓  coupon 2 files
 - <> CouponRateDispatch.sol Solidity
 - <> CouponStorageWrapper.sol Solidity
 - ✓  dividend 1 file
 - <> DividendStorageWrapper.sol Solidity
 - ✓  loan 1 file
 - <> LoanStorageWrapper.sol Solidity
 - ✓  loansPortfolio 2 files
 - <> ILoansPortfolioStorageWrapper.sol Solidity
 - <> LoansPortfolioStorageWrapper.sol Solidity
 - ✓  nominalValue 1 file
 - <> NominalValueStorageWrapper.sol Solidity
 - ✓  types 1 file
 - <> ThirdPartyType.sol Solidity
 - ✓  voting 1 file
 - <> VotingStorageWrapper.sol Solidity
 - <> AdjustBalancesStorageWrapper.sol Solidity
 - <> BondStorageWrapper.sol Solidity
 - <> ClearingStorageWrapper.sol Solidity
 - <> EquityStorageWrapper.sol Solidity
 - <> ERC1410StorageWrapper.sol Solidity
 - <> ERC1594StorageWrapper.sol Solidity
 - <> ERC1644StorageWrapper.sol Solidity
 - <> ERC20PermitStorageWrapper.sol Solidity
 - <> ERC20StorageWrapper.sol Solidity
 - <> ERC20VotesStorageWrapper.sol Solidity
 - <> HoldStorageWrapper.sol Solidity
 - <> InterestRateStorageWrapper.sol Solidity
 - <> KpiLinkedRateLib.sol Solidity
 - <> KpisStorageWrapper.sol Solidity
 - <> LockStorageWrapper.sol Solidity
 - <> ProceedRecipientsStorageWrapper.sol Solidity
 - <> ScheduledTasksStorageWrapper.sol Solidity
 - <> SecurityStorageWrapper.sol Solidity
 - <> SnapshotsStorageWrapper.sol Solidity
 - ✓  core 16 files
 - <> AccessControlStorageWrapper.sol Solidity
 - <> CapStorageWrapper.sol Solidity

- <> ControllistStorageWrapper.sol Solidity
- <> CorporateActionsStorageWrapper.sol Solidity
- <> DeactivateStorageWrapper.sol Solidity
- <> DocumentationStorageWrapper.sol Solidity
- <> ERC3643StorageWrapper.sol Solidity
- <> ExternalListManagementStorageWrapper.sol Solidity
- <> InitializerStorageWrapper.sol Solidity
- <> KycStorageWrapper.sol Solidity
- <> MetadataStorageWrapper.sol Solidity
- <> NonceStorageWrapper.sol Solidity
- <> PauseStorageWrapper.sol Solidity
- <> ProtectedPartitionsStorageWrapper.sol Solidity
- <> ResolverProxyStorageWrapper.sol Solidity
- <> SsiManagementStorageWrapper.sol Solidity
- ✓  orchestrator 8 files
 - <> ClearingLifecycleOps.sol Solidity
 - <> ClearingOps.sol Solidity
 - <> ClearingProtectedOps.sol Solidity
 - <> ClearingReadOps.sol Solidity
 - <> HoldOps.sol Solidity
 - <> ScheduledTasksDispatchOps.sol Solidity
 - <> ScheduledTasksOps.sol Solidity
 - <> TokenCoreOps.sol Solidity
- ✓  facets 363 files
 - ✓  accessControl 3 files
 - <> AccessControl.sol Solidity
 - <> AccessControlFacet.sol Solidity
 - <> IAccessControl.sol Solidity
 - ✓  adjustBalances 3 files
 - <> AdjustBalances.sol Solidity
 - <> AdjustBalancesFacet.sol Solidity
 - <> IAdjustBalances.sol Solidity
 - ✓  allowance 4 files
 - <> Allowance.sol Solidity
 - <> AllowanceFacet.sol Solidity
 - <> IAllowance.sol Solidity
 - <> IAllowanceTypes.sol Solidity
 - ✓  balanceTracker 3 files
 - <> BalanceTracker.sol Solidity
 - <> BalanceTrackerFacet.sol Solidity
 - <> IBalanceTracker.sol Solidity
 - ✓  balanceTrackerAdjusted 3 files
 - <> BalanceTrackerAdjusted.sol Solidity
 - <> BalanceTrackerAdjustedFacet.sol Solidity
 - <> IBalanceTrackerAdjusted.sol Solidity
 - ✓  balanceTrackerAtSnapshot 3 files
 - <> BalanceTrackerAtSnapshot.sol Solidity
 - <> BalanceTrackerAtSnapshotFacet.sol Solidity
 - <> IBalanceTrackerAtSnapshot.sol Solidity
 - ✓  balanceTrackerAtSnapshotByPartition 3 files
 - <> BalanceTrackerAtSnapshotByPartition.sol Solidity

- <> BalanceTrackerAtSnapshotByPartitionFacet.sol Solidity
- <> IBalanceTrackerAtSnapshotByPartition.sol Solidity
- ✓  balanceTrackerByPartition 3 files
 - <> BalanceTrackerByPartition.sol Solidity
 - <> BalanceTrackerByPartitionFacet.sol Solidity
 - <> IBalanceTrackerByPartition.sol Solidity
- ✓  batchBurn 3 files
 - <> BatchBurn.sol Solidity
 - <> BatchBurnFacet.sol Solidity
 - <> IBatchBurn.sol Solidity
- ✓  batchController 3 files
 - <> BatchController.sol Solidity
 - <> BatchControllerFacet.sol Solidity
 - <> IBatchController.sol Solidity
- ✓  batchFreeze 3 files
 - <> BatchFreeze.sol Solidity
 - <> BatchFreezeFacet.sol Solidity
 - <> IBatchFreeze.sol Solidity
- ✓  batchMint 3 files
 - <> BatchMint.sol Solidity
 - <> BatchMintFacet.sol Solidity
 - <> IBatchMint.sol Solidity
- ✓  batchTransfer 3 files
 - <> BatchTransfer.sol Solidity
 - <> BatchTransferFacet.sol Solidity
 - <> IBatchTransfer.sol Solidity
- ✓  burn 3 files
 - <> Burn.sol Solidity
 - <> BurnFacet.sol Solidity
 - <> IBurn.sol Solidity
- ✓  burnByPartition 3 files
 - <> BurnByPartition.sol Solidity
 - <> BurnByPartitionFacet.sol Solidity
 - <> IBurnByPartition.sol Solidity
- ✓  cap 3 files
 - <> Cap.sol Solidity
 - <> CapFacet.sol Solidity
 - <> ICap.sol Solidity
- ✓  capByPartition 3 files
 - <> CapByPartition.sol Solidity
 - <> CapByPartitionFacet.sol Solidity
 - <> ICapByPartition.sol Solidity
- ✓  clearing 3 files
 - <> Clearing.sol Solidity
 - <> ClearingFacet.sol Solidity
 - <> IClearing.sol Solidity
- ✓  clearingAtSnapshot 3 files
 - <> ClearingAtSnapshot.sol Solidity
 - <> ClearingAtSnapshotFacet.sol Solidity
 - <> IClearingAtSnapshot.sol Solidity
- ✓  clearingAtSnapshotByPartition 3 files

- <> ClearingAtSnapshotByPartition.sol Solidity
- <> ClearingAtSnapshotByPartitionFacet.sol Solidity
- <> IClearingAtSnapshotByPartition.sol Solidity
- ✓  clearingByPartition 3 files
 - <> ClearingByPartition.sol Solidity
 - <> ClearingByPartitionFacet.sol Solidity
 - <> IClearingByPartition.sol Solidity
- ✓  clearingHoldByPartition 3 files
 - <> ClearingHoldByPartition.sol Solidity
 - <> ClearingHoldByPartitionFacet.sol Solidity
 - <> IClearingHoldByPartition.sol Solidity
- ✓  compliance 3 files
 - <> Compliance.sol Solidity
 - <> ComplianceFacet.sol Solidity
 - <> IComplianceFacet.sol Solidity
- ✓  complianceByPartition 3 files
 - <> ComplianceByPartition.sol Solidity
 - <> ComplianceByPartitionFacet.sol Solidity
 - <> IComplianceByPartition.sol Solidity
- ✓  controller 3 files
 - <> Controller.sol Solidity
 - <> ControllerFacet.sol Solidity
 - <> IController.sol Solidity
- ✓  controllerByPartition 3 files
 - <> ControllerByPartition.sol Solidity
 - <> ControllerByPartitionFacet.sol Solidity
 - <> IControllerByPartition.sol Solidity
- ✓  controllerHoldByPartition 3 files
 - <> ControllerHoldByPartition.sol Solidity
 - <> ControllerHoldByPartitionFacet.sol Solidity
 - <> IControllerHoldByPartition.sol Solidity
- ✓  controllList 3 files
 - <> ControllList.sol Solidity
 - <> ControllListFacet.sol Solidity
 - <> IControllList.sol Solidity
- ✓  core 3 files
 - <> Core.sol Solidity
 - <> CoreFacet.sol Solidity
 - <> ICore.sol Solidity
- ✓  coreAdjusted 3 files
 - <> CoreAdjusted.sol Solidity
 - <> CoreAdjustedFacet.sol Solidity
 - <> ICoreAdjusted.sol Solidity
- ✓  coreAtSnapshot 3 files
 - <> CoreAtSnapshot.sol Solidity
 - <> CoreAtSnapshotFacet.sol Solidity
 - <> ICoreAtSnapshot.sol Solidity
- ✓  corporateActions 3 files
 - <> CorporateActions.sol Solidity
 - <> CorporateActionsFacet.sol Solidity
 - <> ICorporateActions.sol Solidity

- ✓  coupon 4 files
 - <> Coupon.sol Solidity
 - <> CouponFacet.sol Solidity
 - <> ICoupon.sol Solidity
 - <> ICouponTypes.sol Solidity
- ✓  couponListing 3 files
 - <> CouponListing.sol Solidity
 - <> CouponListingFacet.sol Solidity
 - <> ICouponListing.sol Solidity
- ✓  couponSecurityHolders 3 files
 - <> CouponSecurityHolders.sol Solidity
 - <> CouponSecurityHoldersFacet.sol Solidity
 - <> ICouponSecurityHolders.sol Solidity
- ✓  deactivate 3 files
 - <> Deactivate.sol Solidity
 - <> DeactivateFacet.sol Solidity
 - <> IDeactivate.sol Solidity
- ✓  dividend 4 files
 - <> Dividend.sol Solidity
 - <> DividendFacet.sol Solidity
 - <> IDividend.sol Solidity
 - <> IDividendTypes.sol Solidity
- ✓  dividendSecurityHolders 3 files
 - <> DividendSecurityHolders.sol Solidity
 - <> DividendSecurityHoldersFacet.sol Solidity
 - <> IDividendSecurityHolders.sol Solidity
- ✓  documentation 3 files
 - <> Documentation.sol Solidity
 - <> DocumentationFacet.sol Solidity
 - <> IDocumentation.sol Solidity
- ✓  eip712 3 files
 - <> EIP712.sol Solidity
 - <> EIP712Facet.sol Solidity
 - <> IEIP712.sol Solidity
- ✓  externalControlListManagement 3 files
 - <> ExternalControlListManagement.sol Solidity
 - <> ExternalControlListManagementFacet.sol Solidity
 - <> IExternalControlListManagement.sol Solidity
- ✓  externalKycListManagement 3 files
 - <> ExternalKycListManagement.sol Solidity
 - <> ExternalKycListManagementFacet.sol Solidity
 - <> IExternalKycListManagement.sol Solidity
- ✓  externalPauseManagement 3 files
 - <> ExternalPauseManagement.sol Solidity
 - <> ExternalPauseManagementFacet.sol Solidity
 - <> IExternalPauseManagement.sol Solidity
- ✓  freeze 3 files
 - <> Freeze.sol Solidity
 - <> FreezeFacet.sol Solidity
 - <> IFreeze.sol Solidity
- ✓  freezeAtSnapshot 3 files

- <> FreezeAtSnapshot.sol Solidity
- <> FreezeAtSnapshotFacet.sol Solidity
- <> IFreezeAtSnapshot.sol Solidity
- ✓  freezeAtSnapshotByPartition 3 files
 - <> FreezeAtSnapshotByPartition.sol Solidity
 - <> FreezeAtSnapshotByPartitionFacet.sol Solidity
 - <> IFreezeAtSnapshotByPartition.sol Solidity
- ✓  hold 3 files
 - <> Hold.sol Solidity
 - <> HoldFacet.sol Solidity
 - <> IHoldFacet.sol Solidity
- ✓  holdAtSnapshot 3 files
 - <> HoldAtSnapshot.sol Solidity
 - <> HoldAtSnapshotFacet.sol Solidity
 - <> IHoldAtSnapshot.sol Solidity
- ✓  holdAtSnapshotByPartition 3 files
 - <> HoldAtSnapshotByPartition.sol Solidity
 - <> HoldAtSnapshotByPartitionFacet.sol Solidity
 - <> IHoldAtSnapshotByPartition.sol Solidity
- ✓  holdByPartition 3 files
 - <> HoldByPartition.sol Solidity
 - <> HoldByPartitionFacet.sol Solidity
 - <> IHoldByPartition.sol Solidity
- ✓  identity 3 files
 - <> Identity.sol Solidity
 - <> IdentityFacet.sol Solidity
 - <> IIdentity.sol Solidity
- ✓  initializer 3 files
 - <> IInitializer.sol Solidity
 - <> Initializer.sol Solidity
 - <> InitializerFacet.sol Solidity
- ✓  interestRate 3 files
 - <> IInterestRate.sol Solidity
 - <> InterestRate.sol Solidity
 - <> InterestRateFacet.sol Solidity
- ✓  layer_1 42 files
 - ✓  clearing 4 files
 - ✓  operatorClearingHoldByPartition 3 files
 - <> IOperatorClearingHoldByPartition.sol Solidity
 - <> OperatorClearingHoldByPartition.sol Solidity
 - <> OperatorClearingHoldByPartitionFacet.sol Solidity
 - <> IClearingTypes.sol Solidity
 - ✓  ERC1400 12 files
 - ✓  ERC1410 4 files
 - <> ERC1410Management.sol Solidity
 - <> ERC1410ManagementFacet.sol Solidity
 - <> IERC1410Management.sol Solidity
 - <> IERC1410Types.sol Solidity
 - ✓  ERC20Permit 3 files
 - <> ERC20Permit.sol Solidity
 - <> ERC20PermitFacet.sol Solidity

- <> IERC20Permit.sol Solidity
- ✓  ERC20Votes 5 files
 - <> ERC20Votes.sol Solidity
 - <> ERC20VotesFacet.sol Solidity
 - <> IERC20Votes.sol Solidity
 - <> IERC5805.sol Solidity
 - <> IVotes.sol Solidity
- ✓  ERC3643 7 files
 - <> ERC3643Management.sol Solidity
 - <> ERC3643ManagementFacet.sol Solidity
 - <> ICompliance.sol Solidity
 - <> IERC3643.sol Solidity
 - <> IERC3643Management.sol Solidity
 - <> IERC3643Types.sol Solidity
 - <> IIdentityRegistry.sol Solidity
- ✓  externalControlList 1 file
 - <> IExternalControlList.sol Solidity
- ✓  externalKycList 1 file
 - <> IExternalKycList.sol Solidity
- ✓  externalPause 1 file
 - <> IExternalPause.sol Solidity
- ✓  hold 1 file
 - <> IHoldTypes.sol Solidity
- ✓  kyc 4 files
 - <> IKyc.sol Solidity
 - <> IRevocationList.sol Solidity
 - <> Kyc.sol Solidity
 - <> KycFacet.sol Solidity
- ✓  lock 4 files
 - <> ILock.sol Solidity
 - <> ILockTypes.sol Solidity
 - <> Lock.sol Solidity
 - <> LockFacet.sol Solidity
- ✓  protectedPartition 3 files
 - <> IProtectedPartitions.sol Solidity
 - <> ProtectedPartitions.sol Solidity
 - <> ProtectedPartitionsFacet.sol Solidity
- ✓  snapshot 4 files
 - <> ISnapshots.sol Solidity
 - <> ISnapshotsTypes.sol Solidity
 - <> Snapshots.sol Solidity
 - <> SnapshotsFacet.sol Solidity
- ✓  layer_2 45 files
 - ✓  amortization 4 files
 - ✓  standard 1 file
 - <> AmortizationFacet.sol Solidity
 - <> Amortization.sol Solidity
 - <> AmortizationFacetBase.sol Solidity
 - <> IAmortization.sol Solidity
 - ✓  bond 3 files
 - <> BondRead.sol Solidity

- <> IBondRead.sol Solidity
- <> IBondTypes.sol Solidity
- ✓  equity 2 files
 - <> Equity.sol Solidity
 - <> IEquity.sol Solidity
- ✓  interestRate 7 files
 - ✓  fixedRate 3 files
 - <> FixedRate.sol Solidity
 - <> FixedRateFacet.sol Solidity
 - <> IFixedRate.sol Solidity
 - ✓  kpiLinkedRate 4 files
 - <> IKpiLinkedRate.sol Solidity
 - <> IKpiLinkedRateErrors.sol Solidity
 - <> KpiLinkedRate.sol Solidity
 - <> KpiLinkedRateFacet.sol Solidity
- ✓  kpi/kpiLatest 3 files
 - <> IKpis.sol Solidity
 - <> Kpis.sol Solidity
 - <> KpisKpiLinkedRateFacet.sol Solidity
- ✓  loan 3 files
 - <> ILoan.sol Solidity
 - <> Loan.sol Solidity
 - <> LoanFacet.sol Solidity
- ✓  loansPortfolio 3 files
 - <> ILoansPortfolio.sol Solidity
 - <> LoansPortfolio.sol Solidity
 - <> LoansPortfolioFacet.sol Solidity
- ✓  nominalValue 3 files
 - <> INominalValue.sol Solidity
 - <> NominalValue.sol Solidity
 - <> NominalValueFacet.sol Solidity
- ✓  proceedRecipient 4 files
 - <> IProceedRecipients.sol Solidity
 - <> ProceedRecipients.sol Solidity
 - <> ProceedRecipientsFacet.sol Solidity
 - <> ProceedRecipientsKpiLinkedRateFacet.sol Solidity
- ✓  scheduledTask 6 files
 - ✓  scheduledCrossOrderedTask 4 files
 - <> IScheduledCrossOrderedTasks.sol Solidity
 - <> ScheduledCrossOrderedTasks.sol Solidity
 - <> ScheduledCrossOrderedTasksFacet.sol Solidity
 - <> ScheduledCrossOrderedTasksKpiLinkedRateFacet.sol Solidity
 - ✓  scheduledTasksCommon 1 file
 - <> IScheduledTasksCommon.sol Solidity
 - <> ScheduledTasksLib.sol Solidity
- ✓  security 3 files
 - <> ISecurity.sol Solidity
 - <> Security.sol Solidity
 - <> SecurityFacet.sol Solidity
- ✓  voting 4 files
 - <> IVoting.sol Solidity

- <> IVotingTypes.sol Solidity
- <> Voting.sol Solidity
- <> VotingFacet.sol Solidity
- ✓  layer_3 20 files
 - ✓  bondUSA 10 files
 - ✓  fixedRate 2 files
 - <> BondUSAFixedRateFacet.sol Solidity
 - <> BondUSAReadFixedRateFacet.sol Solidity
 - ✓  kpiLinkedRate 2 files
 - <> BondUSAKpiLinkedRateFacet.sol Solidity
 - <> BondUSAReadKpiLinkedRateFacet.sol Solidity
 - ✓  variableRate 2 files
 - <> BondUSAFacet.sol Solidity
 - <> BondUSAReadFacet.sol Solidity
 - <> BondUSA.sol Solidity
 - <> BondUSAFacetBase.sol Solidity
 - <> BondUSAReadFacetBase.sol Solidity
 - <> IBondUSA.sol Solidity
 - ✓  equityUSA 3 files
 - <> EquityUSA.sol Solidity
 - <> EquityUSAFacet.sol Solidity
 - <> IEquityUSA.sol Solidity
 - ✓  transferAndLock 7 files
 - ✓  fixedRate 1 file
 - <> TransferAndLockFixedRateFacet.sol Solidity
 - ✓  kpiLinkedRate 1 file
 - <> TransferAndLockKpiLinkedRateFacet.sol Solidity
 - ✓  standard 1 file
 - <> TransferAndLockFacet.sol Solidity
 - <> ITransferAndLock.sol Solidity
 - <> ITransferAndLockTypes.sol Solidity
 - <> TransferAndLock.sol Solidity
 - <> TransferAndLockFacetBase.sol Solidity
 - ✓  lockAtSnapshot 3 files
 - <> ILockAtSnapshot.sol Solidity
 - <> LockAtSnapshot.sol Solidity
 - <> LockAtSnapshotFacet.sol Solidity
 - ✓  lockAtSnapshotByPartition 3 files
 - <> ILockAtSnapshotByPartition.sol Solidity
 - <> LockAtSnapshotByPartition.sol Solidity
 - <> LockAtSnapshotByPartitionFacet.sol Solidity
 - ✓  lockByPartition 3 files
 - <> ILockByPartition.sol Solidity
 - <> LockByPartition.sol Solidity
 - <> LockByPartitionFacet.sol Solidity
 - ✓  maturity 3 files
 - <> IMaturity.sol Solidity
 - <> Maturity.sol Solidity
 - <> MaturityFacet.sol Solidity
 - ✓  maturityByPartition 3 files
 - <> IMaturityByPartition.sol Solidity

- <> MaturityByPartition.sol Solidity
- <> MaturityByPartitionFacet.sol Solidity
- ✓  metadata 3 files
 - <> IMetadata.sol Solidity
 - <> Metadata.sol Solidity
 - <> MetadataFacet.sol Solidity
- ✓  mint 3 files
 - <> IMint.sol Solidity
 - <> Mint.sol Solidity
 - <> MintFacet.sol Solidity
- ✓  mintByPartition 3 files
 - <> IMintByPartition.sol Solidity
 - <> MintByPartition.sol Solidity
 - <> MintByPartitionFacet.sol Solidity
- ✓  nominalValueAtSnapshot 3 files
 - <> INominalValueAtSnapshot.sol Solidity
 - <> NominalValueAtSnapshot.sol Solidity
 - <> NominalValueAtSnapshotFacet.sol Solidity
- ✓  nonces 3 files
 - <> INonces.sol Solidity
 - <> Nonces.sol Solidity
 - <> NoncesFacet.sol Solidity
- ✓  operator 3 files
 - <> IOperator.sol Solidity
 - <> Operator.sol Solidity
 - <> OperatorFacet.sol Solidity
- ✓  operatorByPartition 3 files
 - <> IOperatorByPartition.sol Solidity
 - <> OperatorByPartition.sol Solidity
 - <> OperatorByPartitionFacet.sol Solidity
- ✓  operatorClearingByPartition 3 files
 - <> IOperatorClearingByPartition.sol Solidity
 - <> OperatorClearingByPartition.sol Solidity
 - <> OperatorClearingByPartitionFacet.sol Solidity
- ✓  operatorHoldByPartition 3 files
 - <> IOperatorHoldByPartition.sol Solidity
 - <> OperatorHoldByPartition.sol Solidity
 - <> OperatorHoldByPartitionFacet.sol Solidity
- ✓  partitions 3 files
 - <> IPartitions.sol Solidity
 - <> Partitions.sol Solidity
 - <> PartitionsFacet.sol Solidity
- ✓  pause 3 files
 - <> IPause.sol Solidity
 - <> Pause.sol Solidity
 - <> PauseFacet.sol Solidity
- ✓  principal 3 files
 - <> IPrincipal.sol Solidity
 - <> Principal.sol Solidity
 - <> PrincipalFacet.sol Solidity
- ✓  protectedByPartition 3 files

- <> IProtectedByPartition.sol Solidity
- <> ProtectedByPartition.sol Solidity
- <> ProtectedByPartitionFacet.sol Solidity
- ✓  protectedClearingByPartition 3 files
 - <> IProtectedClearingByPartition.sol Solidity
 - <> ProtectedClearingByPartition.sol Solidity
 - <> ProtectedClearingByPartitionFacet.sol Solidity
- ✓  protectedClearingHoldByPartition 3 files
 - <> IProtectedClearingHoldByPartition.sol Solidity
 - <> ProtectedClearingHoldByPartition.sol Solidity
 - <> ProtectedClearingHoldByPartitionFacet.sol Solidity
- ✓  protectedHoldByPartition 3 files
 - <> IProtectedHoldByPartition.sol Solidity
 - <> ProtectedHoldByPartition.sol Solidity
 - <> ProtectedHoldByPartitionFacet.sol Solidity
- ✓  recovery 3 files
 - <> IRecovery.sol Solidity
 - <> Recovery.sol Solidity
 - <> RecoveryFacet.sol Solidity
- ✓  scheduledBalanceAdjustment 3 files
 - <> IScheduledBalanceAdjustment.sol Solidity
 - <> ScheduledBalanceAdjustment.sol Solidity
 - <> ScheduledBalanceAdjustmentFacet.sol Solidity
- ✓  securityHolders 3 files
 - <> ISecurityHolders.sol Solidity
 - <> SecurityHolders.sol Solidity
 - <> SecurityHoldersFacet.sol Solidity
- ✓  securityHoldersAtSnapshot 3 files
 - <> ISecurityHoldersAtSnapshot.sol Solidity
 - <> SecurityHoldersAtSnapshot.sol Solidity
 - <> SecurityHoldersAtSnapshotFacet.sol Solidity
- ✓  snapshotsByPartition 3 files
 - <> ISnapshotsByPartition.sol Solidity
 - <> SnapshotsByPartition.sol Solidity
 - <> SnapshotsByPartitionFacet.sol Solidity
- ✓  ssiManagement 3 files
 - <> ISsiManagement.sol Solidity
 - <> SsiManagement.sol Solidity
 - <> SsiManagementFacet.sol Solidity
- ✓  transfer 3 files
 - <> ITransfer.sol Solidity
 - <> Transfer.sol Solidity
 - <> TransferFacet.sol Solidity
- ✓  transferAndLockByPartition 3 files
 - <> ITransferAndLockByPartition.sol Solidity
 - <> TransferAndLockByPartition.sol Solidity
 - <> TransferAndLockByPartitionFacet.sol Solidity
- ✓  transferByPartition 3 files
 - <> ITransferByPartition.sol Solidity
 - <> TransferByPartition.sol Solidity
 - <> TransferByPartitionFacet.sol Solidity

- ✓  votingSecurityHolders 3 files
 - <> IVotingSecurityHolders.sol Solidity
 - <> VotingSecurityHolders.sol Solidity
 - <> VotingSecurityHoldersFacet.sol Solidity
 - <> IAsset.sol Solidity
- ✓  factory 28 files
- ✓  ERC3643 24 files
 - ✓  interfaces 19 files
 - <> IAccessControl.sol Solidity
 - <> IBondRead.sol Solidity
 - <> IBondTypes.sol Solidity
 - <> IBusinessLogicResolver.sol Solidity
 - <> ICore.sol Solidity
 - <> ICouponListing.sol Solidity
 - <> IDiamondCutManager.sol Solidity
 - <> IDiamondLoupe.sol Solidity
 - <> IEquity.sol Solidity
 - <> IFactory.sol Solidity
 - <> IFixedRate.sol Solidity
 - <> IKpiLinkedRate.sol Solidity
 - <> IKpiLinkedRateErrors.sol Solidity
 - <> IResolverProxy.sol Solidity
 - <> IRexlAccessControl.sol Solidity
 - <> IScheduledTasksCommon.sol Solidity
 - <> IStaticFunctionSelectors.sol Solidity
 - <> regulation.sol Solidity
 - <> roles.sol Solidity
 - ✓  libraries 4 files
 - ✓  core 2 files
 - <> SecurityDeploymentLib.sol Solidity
 - <> TREXBaseDeploymentLib.sol Solidity
 - <> TREXBondDeploymentLib.sol Solidity
 - <> TREXEquityDeploymentLib.sol Solidity
 - <> TREXFactory.sol Solidity
 - <> Factory.sol Solidity
 - <> FactoryFacet.sol Solidity
 - <> IFactory.sol Solidity
 - <> isinValidator.sol Solidity
- ✓  infrastructure 33 files
 - ✓  diamond 9 files
 - <> BusinessLogicResolver.sol Solidity
 - <> BusinessLogicResolverWrapper.sol Solidity
 - <> DiamondCut.sol Solidity
 - <> DiamondCutManager.sol Solidity
 - <> DiamondCutManagerWrapper.sol Solidity
 - <> DiamondFacet.sol Solidity
 - <> DiamondLoupe.sol Solidity
 - <> IBusinessLogicResolver.sol Solidity
 - <> IDiamondCutManager.sol Solidity
- ✓  errors 1 file
 - <> ICommonErrors.sol Solidity

- ✓  proxy 8 files
 - <> Bytes4Builder.sol Solidity
 - <> IDiamond.sol Solidity
 - <> IDiamondCut.sol Solidity
 - <> IDiamondLoupe.sol Solidity
 - <> IResolverProxy.sol Solidity
 - <> IStaticFunctionSelectors.sol Solidity
 - <> ResolverProxy.sol Solidity
 - <> ResolverProxyUnstructured.sol Solidity
- ✓  utils 14 files
 - <> ArrayValidation.sol Solidity
 - <> Checkpoints.sol Solidity
 - <> ContextProvider.sol Solidity
 - <> DatesValidation.sol Solidity
 - <> DecimalsLib.sol Solidity
 - <> DefaultValueValidation.sol Solidity
 - <> EIP712.sol Solidity
 - <> EnumerableSetBytes4.sol Solidity
 - <> EvmAccessors.sol Solidity
 - <> LocalContext.sol Solidity
 - <> LowLevelCall.sol Solidity
 - <> Pagination.sol Solidity
 - <> ProtectedPartitionRoleValidator.sol Solidity
 - <> UnexpectedError.sol Solidity
-  README.md Markdown
- ✓  services 40 files
 - ✓  asset 23 files
 - <> ActionValidationModifiers.sol Solidity
 - <> AdjustBalancesModifiers.sol Solidity
 - <> AmortizationModifiers.sol Solidity
 - <> AssetModifiers.sol Solidity
 - <> BondModifiers.sol Solidity
 - <> ClearingModifiers.sol Solidity
 - <> ComplianceModifiers.sol Solidity
 - <> CouponModifiers.sol Solidity
 - <> EquityModifiers.sol Solidity
 - <> ERC1410Modifiers.sol Solidity
 - <> ERC20Modifiers.sol Solidity
 - <> ERC3643Modifiers.sol Solidity
 - <> ExpirationModifiers.sol Solidity
 - <> HoldModifiers.sol Solidity
 - <> InterestRateModifiers.sol Solidity
 - <> KpisModifiers.sol Solidity
 - <> LoansPortfolioModifiers.sol Solidity
 - <> LockModifiers.sol Solidity
 - <> MaturityModifiers.sol Solidity
 - <> NominalValueModifiers.sol Solidity
 - <> ProceedRecipientModifiers.sol Solidity
 - <> SecurityModifiers.sol Solidity
 - <> StateModifiers.sol Solidity
- ✓  core 15 files

```
<> AccessControlModifiers.sol Solidity
<> CapModifiers.sol Solidity
<> ControllistModifiers.sol Solidity
<> CoreModifiers.sol Solidity
<> DateValidationModifiers.sol Solidity
<> DeactivateModifiers.sol Solidity
<> DefaultValuesModifiers.sol Solidity
<> DocumentationModifiers.sol Solidity
<> ExternalListModifiers.sol Solidity
<> InitializerModifiers.sol Solidity
<> KycModifiers.sol Solidity
<> LoanModifiers.sol Solidity
<> PartitionModifiers.sol Solidity
<> PartitionValidationModifiers.sol Solidity
<> PauseModifiers.sol Solidity
<> InitializationErrors.sol Solidity
<> Modifiers.sol Solidity
```

Out-of-Scope: Third party dependencies and economic attacks.

REMEDIATION COMMIT ID:



```
✧ d41ffd001b732a4d759bfd416231e0e9674f8eef
✧ b94530b5cd9c7189ab8679a435e6d57b2c909f46
✧ 0a574c7ac7c98d675b1b44ac97ef028fa2b69748
✧ 8c7677fe257b44e546190972ee7926094ccf3390
✧ 8c9095a1b07f436eb4c2be5cf4b95f85ce68b89c
✧ 8c9f6fd3d717b04c01e44c9f1b143eb100e7c582
✧ 6a62e06a2e87df77a80cc85c28a7802aa34a81ed
✧ dd1828b7c5510754be8efc2b520599b3eec3a431
✧ def3c9bbab2d129528961819d2b65d084477e9d0
✧ e14641ccffd511fd4b0c98d1f453e7fb47e01dcd
✧ 85e34e15a8a30bcb9b00d2c9891191468eecf714
✧ ecfee8c257bb107af24dfd136be5a8bfad281ed4
✧ be62d3a9a08ceb7047281b7cb4cfd98bbe40217d
✧ 2aa5393d30b88a0dcb37e1d53ed895a63c9dc7ba
✧ c7e2e2c361bb4092865196004085ba1909d9d08e
✧ 28768ee5fcb57bd3bfe5c434805ee47a1365b7b8
✧ b99b6586da0f802356b6ba4a77597fa9936ebb30
✧ 61435bb513dae3dd9a720866685a142c5efe15be
✧ ae11f9fda9c0fb3ef4a9a55c6b2f9fb6dafc230f
✧ 16c7d55322ef1fbf045e2d3c8dc04df7eef2387c
✧ 90847dae60c67127ae76a5c16b51fa6ffdf569b2
```

6701c1fb69d37beda4b40b190ae3fbfd7ee2ba89
e532da9fb15db47eb1e7e542ea8291db019d8cb7
b9aec4f7a2162ad12b00be049dff5f3e92523825
ea626f3dd2c7cbfcd40f42e98b36ebadc2098174
989442bb2b53f90e5e29a7ce2e13f478fa78dcd2
e450e12ccdfb763e7956ac5e39e6e40308743f89
4bda1f8a90b9c93d8a912a84001625f6f62ccd07
1e8b3cb3707ccb2fe7a7c8d1fbe6d25ad7c7c065
96f0781a296bbc2dd83fddf051f2e50769f43c96
5c0caa9688282549e9955877e94b5573b96a3f23
8847126e656eca261927d631b98bb0ff0f2b497b
9879296bb48f533a8c91f45c1c53b42a60fde18f
ccc4716c422da58eda9cc7442bdf7d7ab4fc6d3b
10f49b8530a0c5e7816caedd10cfc76138c07f20

Out-of-Scope: New features/implementations after the remediation commit IDs.

Assessment Summary & Findings Overview

#	TITLE	SEVERITY	SCORE	STATUS
HAL-001	Any caller can freeze and unfreeze holder balances via batch partial entrypoints	High	7.0	SOLVED 08/03/2026
HAL-002	Locker role can move regulated tokens to non-compliant recipients	Medium	5.5	SOLVED 08/12/2026
HAL-003	Third party can hijack facet initializers and inject malicious configuration	Medium	4.7	SOLVED 08/03/2026
HAL-004	Sole-admin recovery without prior role rotation permanently bricks admin operations	Medium	4.7	SOLVED 08/10/2026
HAL-005	Same-timestamp ordering lets a balance adjustment precede a governance snapshot	Low	3.5	SOLVED 08/11/2026

#	TITLE	SEVERITY	SCORE	STATUS
HAL-006	Self-transfer skips the only pause check on several transfer paths	● Low	2.5	SOLVED 08/12/2026
HAL-007	Batch transfer validation ignores cumulative amount and panics mid-execution	● Low	2.5	SOLVED 08/13/2026
HAL-008	Unlimited allowance reverts all spends after a balance adjustment	● Low	2.5	SOLVED 08/13/2026
HAL-009	Wrong KPI window start triggers a false missed-report penalty rate	● Low	2.1	SOLVED 08/13/2026
HAL-010	First KPI coupon with a missed report uses zero instead of the start rate	● Low	2.1	SOLVED 08/03/2026
HAL-011	KPI impact aggregation treats partial reports as complete against one baseline	● Low	2.1	RISK ACCEPTED 07/28/2026
HAL-012	Maturity redemption can strand principal for multi-partition holders	● Low	2.0	SOLVED 08/13/2026
HAL-013	Sole admin can be removed via revokeRole, permanently bricking the proxy	● Informational	1.9	SOLVED 08/13/2026
HAL-014	Revoked KYC credentials stay valid when the revocation registry reverts	● Low	1.9	SOLVED 08/13/2026
HAL-015	Single-step ownership transfer can irrecoverably lock out the factory owner	● Informational	1.5	SOLVED 08/03/2026
HAL-016	Switching rate type away from KPI locks pending coupon rates forever	● Informational	1.0	SOLVED 08/13/2026

#	TITLE	SEVERITY	SCORE	STATUS
HAL-017	Holder can dodge mandatory maturity redemption by lapsing KYC	● Informational	1.0	SOLVED 08/19/2026
HAL-018	Factory deploys partially initialized proxies by swallowing init errors	● Informational	0.9	SOLVED 08/03/2026
HAL-019	Facet-version lifecycle and key-set invariants are not enforced	● Informational	0.8	SOLVED 08/10/2026
HAL-020	Amortization hold release under-subtracts the held balance after an adjustment	● Informational	0.7	SOLVED 09/16/2026
HAL-021	Holder cannot reclaim cleared tokens after their KYC is revoked	● Informational	0.7	SOLVED 08/19/2026
HAL-022	Clearing hold approved after its expiry is created immediately reclaimable	● Informational	0.7	SOLVED 08/18/2026
HAL-023	Operational-status readiness gate is never enforced and any caller can set it	● Informational	0.6	SOLVED 08/03/2026
HAL-024	Coupons can be created with backdated record and fixing dates	● Informational	0.6	SOLVED 08/03/2026
HAL-025	Upgrading the proxy version invalidates all outstanding permit signatures	● Informational	0.5	ACKNOWLEDGED 08/05/2026
HAL-026	Permit and protected operations share one nonce and cross-cancel signatures	● Informational	0.5	SOLVED 08/18/2026
HAL-027	Lock can be created already expired and released in the same block	● Informational	0.5	SOLVED 08/19/2026

#	TITLE	SEVERITY	SCORE	STATUS
HAL-028	Proxy upgrade entrypoints lack target validation and emit no events	● Informational	0.5	SOLVED 08/10/2026
HAL-029	Proceed-recipient init reverts or drops data on array length mismatch	● Informational	0.5	SOLVED 08/19/2026
HAL-030	Reverting loan contract permanently blocks loan-portfolio operations	● Informational	0.4	SOLVED 08/20/2026
HAL-031	Reused batch version returns wrong facet data after configuration cancel	● Informational	0.4	SOLVED 08/10/2026
HAL-032	Factory TREX deploy leaves compliance unbound and can revert the whole suite	● Informational	0.3	SOLVED 08/05/2026
HAL-033	ISIN validator accepts non-alphanumeric characters as valid identifiers	● Informational	0.3	SOLVED 08/06/2026
HAL-034	ERC1594 isIssuable flag is never enforced on any mint path	● Informational	0.3	SOLVED 08/20/2026
HAL-035	Unvalidated KPI bounds cause division-by-zero or underflow, bricking coupons	● Informational	0.2	SOLVED
HAL-036	Scheduled-task dispatch can drop or silently consume a failed task	● Informational	0.1	SOLVED 08/06/2026
HAL-037	Operator clearing cancel and reclaim mint an allowance to address(0)	● Informational	—	SOLVED 09/16/2026
HAL-038	ResolverProxy accepts native HBAR without a rescue path in the current configuration	● Informational	—	ACKNOWLEDGED 09/16/2026

#	TITLE	SEVERITY	SCORE	STATUS
HAL-039	Selector blacklist enforcement is registration-time only and is not documented as such	● Informational	—	SOLVED 09/16/2026

Findings & Tech Details

HAL-001

SOLVED

Any caller can freeze and unfreeze holder balances via batch partial entrypoints

● High · 7.0

A0:A/AC:L/AX:L/R:P/S:C/C:N/A:H/I:H/D:H/Y:N

DESCRIPTION

`BatchFreeze::batchFreezePartialTokens()` and `BatchFreeze::batchUnfreezePartialTokens()` mutate per-holder frozen balances through `ERC3643StorageWrapper::freezeTokens()` and `unfreezeTokens()`, but neither carries the `onlyFreezeRoles` modifier that gates the equivalent single-address functions `Freeze::freezePartialTokens()` and `Freeze::unfreezePartialTokens()`. The only modifiers present are `onlyActivated`, `onlyUnpaused`, `onlyValidInputAmountsArrayLength` and `onlyWithoutMultiPartition`, none of which checks the caller's role, so any account can freeze or unfreeze any holder's balance in bulk.

SOL 46-55

```
46 function batchFreezePartialTokens(  
47     address[] calldata _userAddresses,  
48     uint256[] calldata _amounts  
49 )  
50     external  
51     onlyActivated  
52     onlyUnpaused  
53     onlyValidInputAmountsArrayLength(_userAddresses, _amounts)  
54     onlyWithoutMultiPartition  
55 {
```

Scenario:

1. An unprivileged account calls `batchFreezePartialTokens([victim], [balance])`.
2. The call reaches `ERC3643StorageWrapper::freezeTokens()` with no role check and freezes the victim's tokens, blocking their transfers.
3. The same account calls `batchUnfreezePartialTokens([holder], [amount])` to release a balance an operator had frozen for compliance, defeating the freeze control.

Impact: Any account can freeze any holder's tokens, a denial of service against their transfers, or unfreeze a holder that compliance had frozen, a sanctions-control bypass, both at scale across many holders in a single call.

PROOF OF CONCEPT

The test issues to a victim, then has an account that holds neither `ROLE_FREEZE_MANAGER` nor `ROLE_AGENT` call `BatchFreeze::batchFreezePartialTokens()` and `batchUnfreezePartialTokens()` against that victim.

SOL

```
import { expect } from "chai";
import { ethers } from "hardhat";
import { loadFixture } from "@nomicfoundation/hardhat-network-helpers";
import { deployEquityTokenFixture, executeRbac } from "@test";
import { ATS_ROLES } from "@scripts";

const MAX = ethers.MaxUint256;

describe("PoC: unprivileged batch partial freeze (audited baseline)", () => {
  it("an account with no freeze role freezes and unfreezes a holder in bulk", async () => {
    const { deployer, user1, user2, asset } = await loadFixture(deployEquityTokenFixture);

    // user1 is the attacker (no role); user2 is the victim holder
    await executeRbac(asset, [
      { role: ATS_ROLES.ROLE_ISSUER, members: [deployer.address] },
      { role: ATS_ROLES.ROLE_KYC, members: [deployer.address] },
      { role: ATS_ROLES.ROLE_SSI_MANAGER, members: [deployer.address] },
    ]);
    await asset.connect(deployer).addIssuer(deployer.address);
    await asset.connect(deployer).grantKyc(user2.address, "", 0, MAX, deployer.address);
    await asset.connect(deployer).issue(user2.address, 1000, "0x");

    expect(await asset.hasRole(ATS_ROLES.ROLE_FREEZE_MANAGER, user1.address)).to.be.false;
    expect(await asset.hasRole(ATS_ROLES.ROLE_AGENT, user1.address)).to.be.false;

    // audited baseline: the unprivileged batch call freezes the victim
    await asset.connect(user1).batchFreezePartialTokens([user2.address], [500]);
    expect(await asset.getFrozenTokens(user2.address)).to.equal(500);

    // and clears the freeze just as freely
    await asset.connect(user1).batchUnfreezePartialTokens([user2.address], [500]);
    expect(await asset.getFrozenTokens(user2.address)).to.equal(0);
  });
});
```

Run:

SOL

```
cd packages/ats/contracts
npx hardhat test test/contracts/integration/poc/batch-freeze-no-role.test.ts
```

Output:

SOL

```
PoC: unprivileged batch partial freeze (audited baseline)
  an account with no freeze role freezes and unfreezes a holder in bulk
    user1 holds neither ROLE_FREEZE_MANAGER nor ROLE_AGENT
      batchFreezePartialTokens([user2],[500]) -> getFrozenTokens(user2) == 500
```

```
batchUnfreezePartialTokens([user2],[500]) -> getFrozenTokens(user2) == 0  
1 passing
```

The unprivileged caller, holding neither `ROLE_FREEZE_MANAGER` nor `ROLE_AGENT`, froze the victim's balance to 500 and then cleared it back to 0, freezing and unfreezing holder balances at will.

RECOMMENDATION

Restrict `batchFreezePartialTokens()` and `batchUnfreezePartialTokens()` to freeze-role holders via `onlyFreezeRoles(EvmAccessors.getMsgSender())`, matching the single-address freeze path.

REMEDIATION COMMENT

Addressed in: [d41ffd001b732a4d759bfd416231e0e9674f8eef](#)

SOLVED: The Hashgraph team fixed the vulnerability by adding the freeze-role guard to the public batch partial freeze and unfreeze entrypoints. On the current source the same call reverts with `AccountHasNoRoles` at `BatchFreeze.sol:46` because `onlyFreezeRoles(EvmAccessors.getMsgSender())` now gates both batch partial-freeze entrypoints (PR #1199), confirming the fix.

Locker role can move regulated tokens to non-compliant recipients

● Medium · 5.5

A0:A/AC:L/AX:L/R:P/S:C/C:N/I:H/A:N/D:N/Y:M

DESCRIPTION

`TransferAndLockByPartition::transferAndLockByPartition()` and `TransferAndLock::transferAndLock()` are restricted to `ROLE_LOCKER` and move the caller's tokens to `_to` through `TokenCoreOps.transferByPartition()`, but neither carries the `onlyCanTransferFromByPartition` modifier that gates the standard `TransferByPartition::transferByPartition()` path. That modifier runs `ERC1594StorageWrapper.requireCanTransferFromByPartition()`, which enforces recipient identity (KYC), the compliance module check, and the recovered-wallet check. The lock entrypoints skip it, and the underlying `ERC1410StorageWrapper.transferByPartition()` performs no identity or compliance gate of its own: its post-transfer `ICompliance.transferred()` call is a state-update notification, not an authorisation check, so the recipient is never validated.

SOL 32-56

```

32 function transferAndLockByPartition(
33     bytes32 _partition,
34     address _to,
35     uint256 _amount,
36     bytes calldata _data,
37     uint256 _expirationTimestamp
38 )
39     external
40     override
41     onlyActivated
42     onlyUnpaused
43     onlyRole(ROLE_LOCKER)
44     onlyWithValidExpirationTimestamp(_expirationTimestamp)
45     onlyDefaultPartitionWithSinglePartition(_partition)
46     onlyUnProtectedPartitionsOrWildCardRole
47     returns (uint256 lockId_)
48 {
49     TokenCoreOps.transferByPartition(
50         EvmAccessors.getMsgSender(),
51         IERC1410Types.BasicTransferInfo(_to, _amount),
52         _partition,
53         _data,
54         EvmAccessors.getMsgSender(),
55         ""
56     );
57 }

```

Scenario:

1. An account holding `ROLE_LOCKER` calls `transferAndLockByPartition(partition, to, amount, data, expiration)` with a `to` address that has no KYC claim, fails the compliance module, or is flagged as a recovered wallet.

2. The transfer executes and credits `to` with no recipient validation, then locks the amount until `expiration`.
3. `to` is now a holder of record of the regulated token, appears in the holder directory, and accrues dividend, coupon, and voting snapshots, despite being a recipient the standard transfer path would reject.

Impact: A privileged but compliance-unaware role can seed the regulated security into wallets that fail KYC, compliance, or recovered-status checks, breaking the invariant that every holder is compliance-validated. The illegitimate holder accrues corporate-action entitlements from the moment of receipt, and onward transfer is the only later gate, so the cap table can hold non-compliant positions the rest of the system is built to make impossible.

RECOMMENDATION

Add `onlyCanTransferFromByPartition(EvmAccessors.getMsgSender(), _to, _partition, _amount)` to `transferAndLockByPartition()` and `transferAndLock()` so the lock entrypoints enforce the same recipient identity, compliance, and recovered-wallet checks as `transferByPartition()`.

REMEDATION COMMENT

Addressed in: [🔗 b94530b5cdfc7189ab8679a435e6d57b2c909f46](#)

SOLVED: Standard recipient-compliance validation was added to both transfer-and-lock paths by the `Hashgraph` team. The checks were verified in `TransferAndLock.sol:55-64` and `TransferAndLockByPartition.sol:62-71`, and their use of the common validation was verified in `ERC1594StorageWrapper.sol:212-271`. KYC, identity, control-list, compliance-module, and recovered-wallet restrictions are now enforced before transfer and lock state changes.

Third party can hijack facet initializers and inject malicious configuration

● Medium · 4.7 A0:A/AC:L/AX:M/R:P/S:C/C:N/A:H/I:H/D:H/Y:N

DESCRIPTION

Each facet one-shot initializer (`BondUSA::initializeBondUSA()`, `EquityUSA::initializeEquityUSA()`, `KpiLinkedRate::initializeKpiLinkedRate()`, `NominalValue::initializeNominalValue()` and similar) is protected only by an `onlyNot*Initialized` boolean flag and carries no role gate. The factory drives these through `_tryInitialize*` helpers on the public `Factory::deployProxy()` path. If a facet's initializer is not run (or its run reverts and is swallowed), the initializer stays claimable on a first-come basis by any address.

SOL 10-12

```
10 function initializeBondUSA(  
11     BondDetailsData calldata _bondDetailsData  
12 ) external override onlyNotBondInitialized {
```

Scenario:

1. A bond token is deployed and, for any reason, `initializeBondUSA()` has not yet run on it.
2. A third party watching the public deploy calls `BondUSA::initializeBondUSA()` directly; the only guard, `onlyNotBondInitialized`, is satisfied.
3. The attacker supplies bond detail data of their choosing (nominal value, coupon parameters, recipients).
4. The flag flips to initialised and the legitimate operator can no longer set the real configuration.

Impact: A third party can set coupon rates, nominal value, controllable/forced-transfer flags, proceeds recipients and similar economic configuration to attacker-chosen values on any token whose facet initializer has not yet executed, then lock out the legitimate operator.

RECOMMENDATION

Add an access-control guard to every `initialize*` facet entry point so only the deploying factory or a configured admin can call it, closing the path for a third party to initialize an uninitialized facet.

REMEDIATION COMMENT

Addressed in: [🔗 0a574c7ac7c98d675b1b44ac97ef028fa2b69748](#)

SOLVED: The vulnerable `BondUSA` and `EquityUSA` initializer facets were removed by the `Hashgraph` team. The remaining `NominalValue::initializeNominalValue()` and `KpiLinkedRate::initializeKpiLinkedRate()` paths were verified as restricted by `DEFAULT_ADMIN_ROLE` at `NominalValue.sol:21-33` and `KpiLinkedRate.sol:24-34`. Unprivileged initialization and configuration injection are no longer possible.

Sole-admin recovery without prior role rotation permanently bricks admin operations

● Medium · 4.7 AO:A/AC:L/AX:M/R:P/S:C/C:N/I:H/A:H/D:H/Y:N

DESCRIPTION

The wallet-recovery flow exposed by `Recovery::recoveryAddress()` delegates to `ERC3643StorageWrapper::recoveryAddress()`, which migrates a holder's balance, frozen amount, and control-list membership from a lost or compromised wallet to a new wallet but never migrates AccessControl role memberships. Role state lives in `memberRoles[account]` inside the AccessControl storage, and the recovery body touches only `addressRecovered`, the frozen balance, the transferred balance, and the control list. The pre-condition guard `canRecover()` inspects lock, hold, and clearing balances only and never inspects the role count, so a wallet that holds privileged roles is eligible for recovery while its roles stay bound to the old address. Commit `2303c707d` ("fix(recovery): block recovered wallets in centralized role checks") adds `ERC3643StorageWrapper::checkUnrecoveredAddress(_account)` to the centralised authorization helpers `AccessControlStorageWrapper::checkRole` and `checkAnyRole`, so a recovered wallet is rejected on every role gate that funnels through those helpers, and the commit body explicitly states "Roles are intentionally not migrated". The two design choices combined create a permanent deadlock when a wallet that is the sole holder of `DEFAULT_ADMIN_ROLE` is recovered before that role is rotated to a live address.

SOL 339-366

```
339 function recoveryAddress(  
340     address _lostWallet,  
341     address _newWallet,  
342     address _investorOnchainID,  
343     uint256 _timestamp  
344 ) internal returns (bool) {  
345     ERC3643Storage storage $ = erc3643Storage();  
346     $.addressRecovered[_lostWallet] = true;  
347     $.addressRecovered[_newWallet] = false;  
348     uint256 frozenBalance = getFrozenAmountForAdjustedAt(_lostWallet, _timestamp);  
349     if (frozenBalance > 0) {  
350         unfreezeTokens(_lostWallet, frozenBalance, _timestamp);  
351     }  
352     uint256 balance = ERC1410StorageWrapper.balanceOfAdjustedAt(_lostWallet, _timestamp);  
353     if (balance + frozenBalance > 0) {  
354         ERC20StorageWrapper.transfer(_lostWallet, _newWallet, balance);  
355     }  
356     if (frozenBalance > 0) {  
357         freezeTokens(_newWallet, frozenBalance);  
358     }  
359     if (ControlListStorageWrapper.isInControlList(_lostWallet)) {  
360         ControlListStorageWrapper.addToControlList(_newWallet);  
361     }  
362     emit IERC3643Types.RecoverySuccess(_lostWallet, _newWallet, _investorOnchainID);  
363     return true;  
364 }
```

Scenario:

1. The token is deployed with a single `DEFAULT_ADMIN_ROLE` holder. The factory's minimal-setup default requires at least one `DEFAULT_ADMIN_ROLE` at deployment, so a single admin address is the normal setup, not a rare configuration.
2. The admin wallet `A` is lost or compromised. An operator holding `ROLE_AGENT` follows the documented recovery procedure and calls `recoveryAddress(A, B, investorOnchainID)`. `canRecover(A)` passes because it inspects only lock, hold, and clearing balances.
3. `recoveryAddress()` moves the balance, unfreezes and re-freezes the frozen amount on `B`, copies control-list membership, and sets `addressRecovered[A] = true`. The role storage `memberRoles[A] = {DEFAULT_ADMIN_ROLE}` is left untouched. `B` receives no role.
4. Any call from `A` to a role-gated path reverts with `WalletRecovered` because `checkRole` and `checkAnyRole` now reject a recovered `msg.sender` before the `hasRole` read. `renounceRole` from `A` reverts on `checkNotSoleAdmin`, because `_isSoleAdmin` returns `true` when `roleMembers.length() == 1`. `B` cannot call `grantRole` because `B` does not hold `DEFAULT_ADMIN_ROLE`. No address can grant, revoke, or renounce `DEFAULT_ADMIN_ROLE` from this point forward.

Impact: Every admin-gated path is permanently frozen: `DiamondCut::updateResolver`, `DiamondCut::updateConfig`, `DiamondCut::updateConfigVersion`, `grantRole` and `revokeRole` for every role whose admin is `DEFAULT_ADMIN_ROLE`, and `applyRoles` for every role. The token cannot be upgraded, cannot receive a new operator assignment, and cannot roll back the recovered flag on `A`. The failure surface is availability-critical for the factory's default single-admin deployment, not an edge case, and the trigger is an ordinary operator response to a key compromise.

RECOMMENDATION

Reject recovery of a wallet that still holds any role. Add `getRoleCountFor(_lostWallet) == 0` as a precondition inside `ERC3643StorageWrapper::canRecover()`, or add an equivalent modifier on `RecoveryFacet::recoveryAddress` next to `onlyEmptyWallet(_lostWallet)`. Either placement forces the operator to rotate every role off the lost wallet before recovery, which is the correct incident-response order and closes the sole-admin deadlock at its root: a wallet with no role holdings cannot orphan a role by being recovered.

REMEDIATION COMMENT

Addressed in: [🔗 8c7677fe257b44e546190972ee7926094ccf3390](#)

SOLVED: Recovery of a wallet that still holds an access-control role was prevented by the `Hashgraph` team. The role-count condition was verified in `ERC3643StorageWrapper.sol:521-528`, and maintenance of the reverse role index by the role grant, revoke, and batch-update paths was confirmed. The only administrator can no longer be removed through recovery while the token is left without an administrative recovery path.

Same-timestamp ordering lets a balance adjustment precede a governance snapshot

● Low · 3.5

A0:A/AC:M/AX:M/R:N/S:C/C:N/I:M/A:N/D:N/Y:M

DESCRIPTION

`ScheduledTasksLib::addScheduledTask()` keeps the queue sorted by `scheduledTimestamp` using a strict `<` comparison. Tasks that share a timestamp are therefore ordered relative to each other purely by when they were inserted, not by any deterministic rule tied to their semantic dependency. When a balance adjustment and a governance snapshot are scheduled at the same record-date timestamp, the adjustment can be positioned to execute first, so the snapshot captures balances after the adjustment has already scaled them.

SQL 40-50

```
40         if (_scheduledTasks.scheduledTasks[scheduledTaskPosition].scheduledTimestamp
    < _newScheduledTimestamp) {
    _slideScheduledTasks(_scheduledTasks, scheduledTaskPosition);
    unchecked {
41         ++index;
    }
    } else {
42         newScheduledTaskId = scheduledTaskPosition + 1;
43         _insertScheduledTask(_scheduledTasks, newScheduledTaskId,
44         newScheduledTask);
45         added = true;
46         break;
    }
47     }
```

Scenario:

1. A governance snapshot and a balance adjustment are both scheduled at the same record-date timestamp.
2. Insertion order places the balance-adjustment task ahead of the snapshot task in the queue.
3. At the record-date boundary the queue is flushed; the balance adjustment fires first and scales every holder balance by the adjustment factor.
4. The snapshot then records the post-adjustment balances. Because the snapshot pins the adjustment factor, decimals and nominal value to the same frame and rescales on read, a holder's proportional share is preserved.

Impact: The same-timestamp execution order is decided by insertion order with no semantic guarantee. Today the effect is benign: snapshot reads are adjustment-factor-consistent (balances, decimals and nominal value are pinned to one frame and rescaled on read), so a holder's proportional coupon, dividend and voting entitlement is invariant to whether the snapshot precedes

or follows the adjustment. The risk is latent: a future change that makes snapshot reads frame-dependent would turn this non-deterministic ordering into a real entitlement skew.

RECOMMENDATION

In `ScheduledTasksLib::addScheduledTask()`, break timestamp ties with a deterministic rule that orders governance snapshots ahead of balance adjustments, so a snapshot scheduled at the same record-date timestamp always captures pre-adjustment balances.

REMEDIATION COMMENT

Addressed in: [🔗 8c9095a1b07f436eb4c2be5cf4b95f85ce68b89c](#)

SOLVED: Task-type priority for equal timestamps was added by the `Hashgraph` team. The priority comparison was verified in `ScheduledTasksLib.sol:75-107`, and its use by the cross-ordered queue was verified in `ScheduledTasksStorageWrapper.sol:75-113`. Governance snapshots and coupon listings are now executed before same-time balance adjustments, independent of insertion order.

Self-transfer skips the only pause check on several transfer paths

● Low · 2.5

A0:A/AC:L/AX:L/R:P/S:U/C:N/I:M/A:N/D:N/Y:N

DESCRIPTION

Selectors such as `Transfer::transferWithData()`, `Transfer::transferFromWithData()`, `TransferByPartition::transferByPartition()`, `BurnByPartition::redeemByPartition()` and `OperatorByPartition::operatorTransferByPartition()` declare no `onlyUnpaused` modifier and rely entirely on the cascading pause guard reached through `ERC1410StorageWrapper::beforeTokenTransfer()` -> `triggerAndSyncAll()` -> `ScheduledTasksOps::triggerPendingScheduledCrossOrderedTasks()`, where the lone `PauseStorageWrapper::checkUnpaused()` call lives. `beforeTokenTransfer()` returns immediately when `from == to`, so a self-directed transfer never reaches `triggerAndSyncAll()` and the pause flag is never read.

SQL 542-544

```
542     function beforeTokenTransfer(bytes32 partition, address from, address to, uint256
amount) internal {
        if (from == to) return;
543         triggerAndSyncAll(partition, from, to);
544     }
```

Scenario:

1. An operator triggers `Pause::pause()` during an incident.
2. A holder calls `Transfer::transferWithData(self, amount, "0x")` with `to` equal to their own address.
3. `beforeTokenTransfer()` sees `from == to` and returns before `triggerAndSyncAll()`, so `checkUnpaused()` never runs.
4. The transfer settles. Because the same self-address path also skips the scheduled-task flush, due corporate actions (balance adjustments, snapshots) are silently deferred while the holder believes the asset is frozen.

Impact: Any selector that depends on the cascading guard rather than an explicit `onlyUnpaused` can be executed during an emergency pause by setting `from == to`, and the same shortcut skips the scheduled-task and balance-adjustment flush, leaving due corporate actions unprocessed.

RECOMMENDATION

Add a `checkUnpaused()` call to `ERC1410StorageWrapper::beforeTokenTransfer()` before the `from == to` early return, and declare `onlyUnpaused` on `Transfer::transferWithData()`,

`Transfer::transferFromWithData()`, `Burn::redeem()` and `Burn::redeemFrom()`.

REMEDIATION COMMENT

Addressed in: [🔗 8c9f6fd3d717b04c01e44c9f1b143eb100e7c582](#)

SOLVED: The pause check was moved ahead of the self-transfer return, and explicit guards were added by the `Hashgraph` team. The central check was verified in

`ERC1410StorageWrapper.sol:552-556`, and the affected entry points were verified in

`Transfer.sol:70-104`, `Burn.sol:50-81`, `TransferByPartition.sol:31-49`, and

`OperatorByPartition.sol:72-90`. The paused state can no longer be bypassed through self-transfers.

Batch transfer validation ignores cumulative amount and panics mid-execution

● Low · 2.5

A0:A/AC:L/AX:L/R:P/S:U/C:N/I:M/A:N/D:N/Y:N

DESCRIPTION

`BatchTransfer::batchTransfer()` validates in one loop and executes in another. The validation loop calls `requireCanTransferFromByPartition()` for each entry, checking the individual `_amounts[i]` against the sender's full balance, never against a running total. When the cumulative batch amount exceeds the balance but each individual amount does not, validation passes. The execution loop then deducts sequentially until a checked subtraction underflows and panics mid-batch.

SOL 36-52

```
36 for (uint256 i; i < length; ) {
37     ERC1594StorageWrapper.requireCanTransferFromByPartition(
38         EvmAccessors.getMsgSender(),
39         _toList[i],
40         _DEFAULT_PARTITION,
41         _amounts[i]
42     );
43     unchecked {
44         ++i;
45     }
46 }
47 for (uint256 i; i < length; ) {
48     TokenCoreOps.transfer(EvmAccessors.getMsgSender(), _toList[i], _amounts[i]);
49     unchecked {
50         ++i;
51     }
52 }
```

Scenario:

1. A sender holds 100 tokens and submits a batch of three transfers of 60, 50 and 40.
2. The validation loop checks each amount against the full 100 balance, so all three pass.
3. The execution loop transfers 60, then attempts 50 against the remaining 40 and reverts with an underflow panic after the first transfer already executed.
4. A prior `canTransfer`-style query over the same amounts returns success, misleading the caller before the on-chain revert.

Impact: Batch transfers whose cumulative amount exceeds the balance pass validation and then panic mid-execution, and the per-element `canTransfer` view misreports such a batch as transferable.

RECOMMENDATION

Accumulate the per-recipient amounts in the validation loop of `batchTransfer()` and verify the running total against the sender's balance, so an over-balance batch is rejected up front rather than panicking mid-execution.

REMIEDIATION COMMENT

Addressed in: [🔗 6a62e06a2e87df77a80cc85c28a7802aa34a81ed](#)

SOLVED: Checked cumulative debit validation and consistent self-transfer handling were added by the `Hashgraph` team. The validation and execution loops were verified in `BatchTransfer.sol:38-92`, and the balance checks were verified in `TokenCoreOps.sol:117-124` and `ERC20StorageWrapper.sol:327-347`. The total can no longer be wrapped by a batch or fail only after an earlier debit, and self-transfer legs remain no-ops.

Unlimited allowance reverts all spends after a balance adjustment

Low · 2.5

A0:A/AC:L/AX:L/R:P/S:U/C:N/I:N/A:M/D:N/Y:N

DESCRIPTION

`ERC20StorageWrapper::updateAllowanceAndLabaf()` scales a stored allowance by the adjustment catch-up factor with a checked multiply, `allowed[owner][spender] *= calculateFactor(abaf, labaf)`. When the stored allowance is `type(uint256).max`, the common unlimited-approval pattern, any adjustment with a factor above 1 makes this multiply overflow and revert with `Panic(0x11)`. The function runs inside `beforeAllowanceUpdate()`, which executes before every `decreaseAllowedBalance()`, so `transferFrom()`, `decreaseAllowance()` and `burnFrom()` all revert for that pair.

SOL 218-226

```

218 function updateAllowanceAndLabaf(address owner, address spender) internal {
219     uint256 abaf = AdjustBalancesStorageWrapper.getAbaf();
220     uint256 labaf = AdjustBalancesStorageWrapper.getAllowanceLabaf(owner, spender);
221
222     if (abaf == labaf) return;
223
224     erc20Storage().allowed[owner][spender] *=
    AdjustBalancesStorageWrapper.calculateFactor(abaf, labaf);
    AdjustBalancesStorageWrapper.updateAllowanceLabaf(owner, spender, abaf);
    }

```

Scenario:

1. An owner grants an unlimited allowance of `type(uint256).max` to a spender.
2. A balance adjustment with factor greater than 1 occurs.
3. The spender calls `transferFrom()`. `beforeAllowanceUpdate()` runs `updateAllowanceAndLabaf()`, which multiplies `type(uint256).max` by the factor and reverts with an overflow panic.
4. Every subsequent `transferFrom()`, `decreaseAllowance()` and `burnFrom()` for that pair reverts the same way.

Impact: After any factor-greater-than-1 adjustment, `transferFrom()`, `decreaseAllowance()`, `burnFrom()` and the `allowance()` view all revert for an owner and spender holding an unlimited approval, until the owner calls `approve()` again, which re-anchors the allowance to the current factor and clears the condition. The break is recoverable, but it is triggered by a routine corporate action against the most common approval value.

RECOMMENDATION

In `ERC20StorageWrapper::updateAllowanceAndLabaf()`, skip the catch-up multiply when the stored allowance is `type(uint256).max`, treating it as an unlimited-approval sentinel.

REMEDIATION COMMENT

Addressed in: [🔗 dd1828b7c5510754be8efc2b520599b3eec3a431](#)

SOLVED: Unlimited allowances were preserved across ABAF updates, and conditional debit and restoration paths were corrected by the `Hashgraph` team. The allowance helpers were verified in `ERC20StorageWrapper.sol:381-414`, hold handling was verified in `HoldStorageWrapper.sol:150-159` and `HoldStorageWrapper.sol:895-908`, and clearing handling was verified in `ClearingOps.sol:306-321` and `ClearingLifecycleOps.sol:345-374`. An overflow can still occur when a near-maximum finite allowance is increased before restoration; this rare, recoverable boundary is an accepted low-impact residual.

HAL-009

SOLVED

Wrong KPI window start triggers a false missed-report penalty rate

● Low · 2.1

A0:A/AC:L/AX:H/R:N/S:U/C:N/I:M/A:N/D:L/Y:L

DESCRIPTION

`KpiLinkedRateLib::_collectImpactData()` computes the lookback window start as `fixingDate > reportPeriod ? fixingDate - reportPeriod : fixingDate`. When `fixingDate <= reportPeriod` the else branch sets `windowStart = fixingDate`, which is the end of the window rather than its start. The subsequent `getLatestKpiData(windowStart, fixingDate, ...)` lookup then spans an empty or near-empty range, so genuine KPI reports are not found and `reportFound_` stays false.

SOL

```
uint256 windowStart;
unchecked {
    windowStart = fixingDate > reportPeriod ? fixingDate - reportPeriod : fixingDate;
}
```

Scenario:

1. A KPI-linked coupon is configured with `fixingDate <= reportPeriod` (an early coupon, short reporting period).
2. A valid KPI report exists for the project before `fixingDate`.
3. `_collectImpactData` sets `windowStart = fixingDate`, so the report falls outside the queried range.
4. The function reports no data, and rate resolution falls into the missed-report path, applying the penalty rate.

Impact: Coupons whose fixing date is within one report period of the window are wrongly treated as missing their KPI report, so holders receive the penalty rate instead of the rate their actual KPI performance earned.

RECOMMENDATION

In `KpiLinkedRateLib::_collectImpactData()`, set `windowStart` to `0` rather than `fixingDate` when `fixingDate` is at or before the report period, so the lookback covers the full period and a valid report is found.

REMEDIATION COMMENT

Addressed in: [🔗 def3c9bbab2d129528961819d2b65d084477e9d0](#)

SOLVED: The early lookback window was corrected by the `Hashgraph` team. A zero lower bound when `fixingDate <= reportPeriod` was verified in `KpiLinkedRateLib.sol:111-124`, and both boundary cases were verified in `kpiLinkedRate.test.ts:698-766`. Reports from the full valid period can now be included in the lookup.

First KPI coupon with a missed report uses zero instead of the start rate

● Low · 2.1

A0:A/AC:L/AX:H/R:N/S:U/C:N/I:M/A:N/D:L/Y:L

DESCRIPTION

When a KPI-linked coupon has no report, `KpiLinkedRateLib::_getRateWhenNoReport()` derives the rate from the previous coupon's rate via `_previousRate()`. For the first coupon, `getPreviousCouponInOrderedList()` returns `0` and `_previousRate()` returns `(0, 0)`. The no-report rate then becomes `0 + missedPenalty` rather than `startRate + missedPenalty`, so the first coupon ignores the configured starting rate.

SOL 110-114

```
110         uint256 previousCouponId =  
            CouponStorageWrapper.getPreviousCouponInOrderedList(couponID);  
  
111         if (previousCouponId == 0) {  
112             return (0, 0);  
113         }  
114     }
```

Scenario:

1. A KPI-linked bond is configured with a non-zero `startRate`.
2. The first coupon period passes with no KPI report submitted.
3. Rate resolution calls `_getRateWhenNoReport`, which reads a previous rate of `0`.
4. The first coupon pays `missedPenalty` off a zero base instead of off the start rate.

Impact: The first KPI coupon under a missed report is computed from zero rather than the configured start rate, understating the coupon owed to holders for that period.

RECOMMENDATION

Return the configured `startRate` (and its decimals) as the previous-rate fallback for the first coupon in `_previousRate()`, so a missed first report bases the rate on the start rate.

REMEDIATION COMMENT

Addressed in: [e14641ccffd511fd4b0c98d1f453e7fb47e01dcd](#)

SOLVED: The Hashgraph team added an explicit predecessor-found flag in commit [e14641ccf](#) (PR #1218). `KpiLinkedRateLib::_previousRate()` returns `found_ = false` when no prior coupon exists at `KpiLinkedRateLib.sol:147-162`, and `_getRateWhenNoReport()` then uses `baseRate`

before adding the missed-report penalty at `KpiLinkedRateLib.sol:79-98`. The first KPI coupon therefore no longer calculates its missed-report rate from zero.

KPI impact aggregation treats partial reports as complete against one baseline

● Low · 2.1

AO:A/AC:L/AX:H/R:N/S:U/C:N/A:N/I:M/D:L/Y:L

DESCRIPTION

`KpiLinkedRateLib::_collectImpactData()` iterates the proceed recipients and accumulates each one's latest KPI value into `impactData_`, marking `reportFound_` true as soon as any single recipient has data. The aggregated sum is later compared against one configured `baseLine` in `_getRateFromImpact()`. If only a subset of recipients reported, the partial sum is treated as a complete aggregate against the same baseline, and the baseline is fixed regardless of how many recipients exist.

SQL 87-96

```
87         for (uint256 index; index < projectCount; ) {
88             address[] memory projects =
ProceedRecipientsStorageWrapper.getProceedRecipients(index, 1);
89             (uint256 value, bool exists) = KpisStorageWrapper.getLatestKpiData(windowStart,
90 fixingDate, projects[0]);
91             if (exists) {
92                 impactData_ += value;
93                 reportFound_ = true;
94             }
95         }
```

Scenario:

1. A KPI configuration has several proceed recipients and one shared baseline.
2. Only some recipients submit KPI data for the period.
3. `_collectImpactData` sums the reported values and flags a report found.
4. The partial sum is compared against the full baseline, skewing the rate toward a missed/under-performing result.

Impact: Partial KPI reporting is scored as if every recipient reported, and the single baseline drifts relative to actual coverage, producing coupon rates that do not reflect true aggregate KPI performance.

RECOMMENDATION

In `KpiLinkedRateLib::_collectImpactData()`, resolve the rate only when every proceed recipient has reported for the window and otherwise route to the missed-report path, so a partial sum is never scored against the full baseline.

REMEDIATION COMMENT

RISK ACCEPTED: Partial KPI reports continue to be aggregated against the shared baseline. This behavior was verified in `KpiLinkedRateLib.sol:119-129`, where a report from any recipient sets `reportFound_`, and in `KpiLinkedRateLib.sol:183-187`, where the partial aggregate is compared with the configured baseline. Per-recipient reporting was retained as an off-chain process requirement, and no contract change was planned by the `Hashgraph` team. The following statement was provided by the `Hashgraph` team:

From our client perspective:

- Several proceed recipients can report against the same shared KPI baseline. In a given period, one or more recipients may not submit a KPI report; `_collectImpactData()` deliberately excludes a non-reporting recipient from the aggregate rather than treating the absence as an error.
 - This is not an oversight: it is how the end client defined the requirement, and the client has raised no objection to this behaviour.
 - Client's own functional contact separately indicated that a recipient is expected to report for every coupon period; where a recipient has nothing new to report, the expectation is that they resubmit their last known value rather than skip the period. That expectation lives on the reporting-process side (the recipients' responsibility), not in the contract — the contract's role is limited to aggregating whatever reports it receives at the time rate resolution runs.
 - Conclusion: functionally correct as implemented, at this time. No code change is planned for this finding; if the client's expectations around mandatory per-period reporting change in future, this should be revisited then, not now.
-

Maturity redemption can strand principal for multi-partition holders

● Low · 2.0

A0:A/AC:M/AX:M/R:P/S:U/C:N/I:N/A:M/D:H/Y:N

DESCRIPTION

`Maturity::fullRedeemAtMaturity()` reads the full partition list via `partitionsOf(_tokenHolder)` and redeems each partition in a single unbounded loop. The number of partitions is not capped, and partitions can be created by ordinary transfer activity. Once a holder owns enough partitions, the loop's gas cost exceeds the block gas limit and the transaction can never succeed.

SOL 42-54

```
42         bytes32[] memory partitions = ERC1410StorageWrapper.partitionsOf(_tokenHolder);
         uint256 length = partitions.length;
         address sender = EvmAccessors.getMsgSender();
43         for (uint256 i; i < length; ) {
44             bytes32 partition = partitions[i];
             uint256 balance = ERC1410StorageWrapper.balanceOfByPartition(partition,
45 _tokenHolder);
46             if (balance != 0) {
                 _redeemByPartition(partition, _tokenHolder, sender, balance);
47             }
             unchecked {
                 ++i;
48             }
49         }
```

Scenario:

1. A holder accumulates a large number of partitions (through normal partitioned transfers or deliberate partition spam).
2. The bond reaches maturity and `ROLE_MATURITY_REDEEMER` calls `fullRedeemAtMaturity(holder)`.
3. The loop iterates every partition; total gas exceeds the block gas limit and the call reverts.
4. There is no paginated alternative, so the holder's principal cannot be redeemed at maturity.

Impact: Mandatory maturity redemption can be blocked for any holder with enough partitions, permanently stranding their principal at maturity with no recovery path through this function.

RECOMMENDATION

Add a paginated redemption entry point that processes partitions in bounded batches (caller-supplied offset and count), so redemption completes within the block gas limit regardless of partition count.

REMEDIATION COMMENT

Addressed in: [85e34e15a8a30bcb9b00d2c9891191468eecf714](#)

SOLVED: Bounded maturity redemption was added by the [Hashgraph](#) team. Requested-range-only processing was verified in [Maturity.sol:138-171](#), and safe high-to-low swap-and-pop removal was verified in [ERC1410StorageWrapper.sol:1145-1167](#). Page zero must be repeated for complete processing as the array is compacted; this is an accepted operational limitation.

Sole admin can be removed via revokeRole, permanently bricking the proxy

• Informational · 1.9 AO:S/AC:L/AX:L/R:N/S:C/C:N/I:N/A:H/D:N/Y:N

DESCRIPTION

The sole-admin guard `AccessControlStorageWrapper::checkNotSoleAdmin()` is enforced only on `AccessControl::renounceRole()`, `AccessControl::revokeRole()` and `AccessControl::applyRoles()` can both drop a role from any account, including the last holder of `DEFAULT_ADMIN_ROLE`, with no equivalent check. `revokeRole()` requires only that the caller hold the role's admin role, and `DEFAULT_ADMIN_ROLE` is its own admin, so a sole admin can revoke itself.

SOL 43-54

```
43 function revokeRole(  
44     bytes32 _role,  
45     address _account  
46 )  
47     external  
48     override  
49     onlyActivated  
50     onlyUnpaused  
51     onlyRole(AccessControlStorageWrapper.getRoleAdmin(_role))  
52     returns (bool success_)  
53 {  
54     success_ = AccessControlStorageWrapper.revokeRole(_role, _account);  
55 }
```

Scenario:

1. A token has exactly one `DEFAULT_ADMIN_ROLE` holder.
2. That holder (by mistake, a malicious operator, or a compromised key) calls `AccessControl::revokeRole(DEFAULT_ADMIN_ROLE, self)`.
3. `revokeRole()` passes the `onlyRole` admin check and removes the membership with no `checkNotSoleAdmin` guard.
4. The token now has zero admins. Every admin-gated operation (role management, configuration, pause) is permanently unreachable, and `applyRoles()` offers the same path.

Impact: The proxy is bricked at the access-control layer with no recovery path. No account can grant, revoke, or manage roles again, freezing every privileged function for the lifetime of the token.

RECOMMENDATION

Call `checkNotSoleAdmin(_role)` inside `AccessControl::revokeRole()` and `AccessControl::applyRoles()` for any path that removes `DEFAULT_ADMIN_ROLE`, matching the

guard already present on `renounceRole()`.

REMEDIATION COMMENT

Addressed in: [🔗 ecfee8c257bb107af24dfd136be5a8bfad281ed4](#)

SOLVED: Single and batch role revocations were routed through the shared sole-administrator guard by the `Hashgraph` team. The common revocation path was verified in

`AccessControlStorageWrapper.sol:98-131`, the sole-administrator check was verified in

`AccessControlStorageWrapper.sol:322-330`, and the external role path was verified in

`AccessControlBase.sol:79-85`.

Revoked KYC credentials stay valid when the revocation registry reverts

● Low · 1.9 A0:A/AC:L/AX:H/R:P/S:C/C:N/I:H/A:H/D:N/Y:N

DESCRIPTION

`KycStorageWrapper::getKycStatusFor()` queries the optional SSI revocation registry with `try IRevocationList(...).revoked(issuer, vcId)`. On a revert the bare `catch` block does nothing and execution falls through to `return kycFor.status`, which is the stored GRANTED value. A registry that reverts (paused, self-destructed, out of gas, or returning malformed data) therefore leaves a credential effectively granted even when it has actually been revoked.

SOL 131-141

```
131         if (revocationListAddress != address(0)) {  
132             try IRevocationList(revocationListAddress).revoked(kycFor.issuer, kycFor.vcId)  
133             returns (bool revoked) {  
134                 if (revoked) {  
135                     return IKyc.KycStatus.NOT_GRANTED;  
136                 }  
137             } catch {  
138                 // we consider that the kyc was not revoked  
139             }  
140         }  
141         return kycFor.status;
```

Scenario:

1. A holder's verifiable credential is revoked on the SSI revocation registry.
2. The registry contract reverts on calls (paused, removed, or returns malformed data).
3. `getKycStatusFor()` calls `revoked()`, the `try` reverts, and the empty `catch` swallows it.
4. The function returns the stored GRANTED status, so the revoked holder passes every KYC-gated transfer and compliance check.

Impact: While a configured revocation registry is unreachable or reverting, a credential revoked on that registry is still read as GRANTED, so the check fails open for the duration of the outage and admits a holder who should be blocked from regulated transfers.

RECOMMENDATION

Treat a failed revocation-registry call as revoked (or revert): return `NOT_GRANTED` in the `catch` block so an unreachable revocation source closes the gate rather than opening it.

REMEDIATION COMMENT

Addressed in: [🔗 be62d3a9a08ceb7047281b7cb4cfd98bbe40217d](#)

SOLVED: The revocation-registry failure path was changed to return `NOT_GRANTED` by the `Hashgraph` team. The fail-closed result was verified in `KycStorageWrapper.sol:121-140`, and its regression coverage was verified in `ssi.test.ts:195-224`. A stored `GRANTED` status can no longer be preserved by a failed or unreachable registry.

Single-step ownership transfer can irrecoverably lock out the factory owner

• Informational · 1.5 AO:S/AC:L/AX:L/R:N/S:U/C:N/I:N/A:H/D:N/Y:N

DESCRIPTION

`TREXFactoryAts` inherits OpenZeppelin `Ownable`, whose `transferOwnership()` transfers control in a single step with immediate effect and no pending-owner acceptance.

`TREXFactoryAts::recoverContractOwnership()` likewise calls

`Ownable(_contract).transferOwnership(_newOwner)` on external infrastructure in one step. A wrong, compromised or unreachable target address therefore takes ownership immediately with no path to reclaim it.

SOL 19-23

```
19 contract TREXFactoryAts is ITREXFactory, Ownable {  
    ...  
20     function recoverContractOwnership(address _contract, address _newOwner) external  
21     override onlyOwner {  
        (Ownable(_contract)).transferOwnership(_newOwner);  
    }  
}
```

Scenario:

1. The owner calls `transferOwnership()` with a mistyped or wrong-key address.
2. Because the transfer is single-step, ownership moves to that address immediately.
3. The intended owner can no longer call `onlyOwner` functions, and there is no two-step acceptance that would have caught the error.
4. The same hazard applies when `recoverContractOwnership()` hands an external contract's ownership to a wrong address.

Impact: A single mistyped owner address irrecoverably loses control of the factory or of external infrastructure it administers, with no pending-owner safeguard.

RECOMMENDATION

Adopt OpenZeppelin `Ownable2Step` for `TREXFactoryAts` so `transferOwnership()` requires the new owner to accept, and route `recoverContractOwnership()` through the same two-step acceptance.

REMEDIATION COMMENT

Addressed in: [🔗 2aa5393d30b88a0dcb37e1d53ed895a63c9dc7ba](#)

SOLVED: The Hashgraph team removed `TREXFactoryAts` and the complete `packages/ats/contracts/contracts/factory/ERC3643/` implementation in commit `2aa5393d3` (PR #1341). The single-step ownership-transfer function and its irrecoverable acceptance path are no longer present on `origin/development`, removing the affected surface and confirming the fix by removal.

Switching rate type away from KPI locks pending coupon rates forever

• Informational · 1.0 AO:S/AC:L/AX:M/R:N/S:U/C:N/I:M/A:M/D:L/Y:L

DESCRIPTION

`CouponRateDispatch::resolveRate()` computes a KPI-linked rate only when the current rate type is `KPI_LINKED`; for every other type it returns `shouldOverride_ = false` and leaves the rate untouched. Coupons created while the type is `KPI_LINKED` are stamped PENDING and rely on this resolution at trigger time. `InterestRate::setCouponRateType()` can move the type away from `KPI_LINKED`, after which the pending coupons never get a resolved rate.

SOL 47-51

```
47         if (InterestRateStorageWrapper.getCouponRateType() ==  
            IInterestRate.RateType.KPI_LINKED) {  
                (rate_, rateDecimals_) =  
                KpiLinkedRateLib.calculateKpiLinkedInterestRate(couponID, coupon);  
48         return (rate_, rateDecimals_, true);  
        }  
        return (rate_, rateDecimals_, shouldOverride_);  
49
```

Scenario:

1. The token is configured `KPI_LINKED` and several coupons are issued PENDING.
2. A holder of `ROLE_INTEREST_RATE_MANAGER` calls `setCouponRateType` with a non-KPI type.
3. At each pending coupon's trigger, `resolveRate` takes the non-KPI branch and returns `shouldOverride_ = false`.
4. The pending coupons keep their unresolved (zero) rate permanently, paying nothing.

Impact: A rate-type change after KPI coupons are issued permanently strands those coupons at their unresolved rate, silently destroying the payable amount holders were owed.

RECOMMENDATION

In `InterestRate::setCouponRateType()`, resolve and stamp all pending KPI coupon rates before changing the type away from `KPI_LINKED`, so no coupon is left at an unresolved rate.

REMEDIATION COMMENT

Addressed in: [c7e2e2c361bb4092865196004085ba1909d9d08e](#)

SOLVED: The historical scan was replaced with a bounded pending-coupon set by the `Hashgraph` team. Set maintenance on creation, resolution, and cancellation was verified in `CouponStorageWrapper.sol:40-48`, `CouponStorageWrapper.sol:72-122`, and

`CouponStorageWrapper.sol:363-381`; the rate-type check is now constant time. Coupons that were pending before the set existed must be resolved, cancelled, or backfilled during stateful upgrades.

Holder can dodge mandatory maturity redemption by lapsing KYC

• Informational · 1.0 AO:S/AC:L/AX:L/R:N/S:U/C:N/I:N/A:M/D:N/Y:N

DESCRIPTION

`Maturity::fullRedeemAtMaturity()` gates mandatory maturity redemption on `onlyListedAllowed(_tokenHolder)` and `onlyValidKycStatus(IKyc.KycStatus.GRANTED, _tokenHolder)`, both evaluated against the holder at redemption time. A bond holder can defeat the redemption by letting their KYC lapse before maturity or by ending up on the control list. When `ROLE_MATURITY_REDEEMER` calls the function, these modifiers revert and the holder's bond tokens stay unredeemed.

SOL 27-41

```
27 function fullRedeemAtMaturity(  
28     address _tokenHolder  
29 )  
30     external  
31     override  
32     onlyActivated  
33     onlyUnpaused  
34     onlyClearingDisabled  
35     onlyRole(ROLE_MATURITY_REDEEMER)  
36     onlyValidAddress(_tokenHolder)  
37     onlyUnrecoveredAddress(_tokenHolder)  
38     onlyListedAllowed(_tokenHolder)  
39     onlyValidKycStatus(IKyc.KycStatus.GRANTED, _tokenHolder)  
40     onlyValidMaturityDate(TimeTravelStorageWrapper.getBlockTimestamp())  
41 {
```

Scenario:

1. A holder owns bond tokens approaching maturity.
2. The holder lets their KYC expire, or arranges to be blacklisted, before the maturity date.
3. `ROLE_MATURITY_REDEEMER` calls `fullRedeemAtMaturity()`, which reverts on `onlyValidKycStatus` or `onlyListedAllowed`.
4. The holder's bond tokens remain unredeemed and the issuer cannot complete the maturity settlement for that holder.

Impact: A holder can unilaterally block the mandatory maturity settlement the bond lifecycle requires by dropping their own KYC or control-list status, and no redeemer-side override exists to force completion, so an obligatory corporate action on a regulated instrument can be left permanently incomplete for that holder at their own discretion.

RECOMMENDATION

Exempt `Maturity::fullRedeemAtMaturity()` from the holder KYC and control-list modifiers so issuer-driven mandatory settlement cannot be blocked by the holder, and document the carve-out.

REMEDIATION COMMENT

Addressed in: [🔗 28768ee5fcb57bd3bfe5c434805ee47a1365b7b8](#)

SOLVED: Holder KYC and control-list gates were removed from issuer-driven maturity redemption by the `Hashgraph` team. Both entry points were verified in `Maturity.sol:42-85` and `Maturity.sol:122-173`, and the retained transfer controls were verified in `TokenCoreOps.sol:85-101` and `ERC1410StorageWrapper.sol:427-453`. Role, recovery, maturity, pause, clearing, activation, and operational checks remain enforced.

Factory deploys partially initialized proxies by swallowing init errors

• Informational · 0.9 AO:A/AC:L/AX:H/R:P/S:U/C:N/I:M/A:L/D:N/Y:N

DESCRIPTION

Every facet initialisation on `Factory::deployProxy()` is routed through a `_tryInitialize*` helper that uses `try ... catch {}` with an empty catch block. The catch swallows all revert data (out-of-gas, malformed parameters, a genuinely reverting initializer) and the factory continues. No event records that the facet was skipped.

SOL 378-384

```
378     function _tryInitialize_ERC1594(address securityAddress_) private {  
379         try IMint(securityAddress_).initialize_ERC1594() {  
380             // success  
381         } catch {  
382             // facet not present - skip initialization  
383         }  
384     }
```

Scenario:

1. A token is deployed; one facet's initializer reverts during `deployProxy()` (bad input, gas, or a transient condition).
2. The surrounding `_tryInitialize*` helper catches the revert, discards the reason, and the deploy transaction succeeds.
3. The proxy now exists with that facet uninitialised and its `onlyNot*Initialized` flag still false.
4. A third party calls the facet initializer directly and sets its configuration, because the only guard is the unset flag.

Impact: A failed facet initialisation during deployment is swallowed with no event or revert, so deployment cannot signal partial failure; an initializer left unrun keeps its `onlyNot*Initialized` flag false and stays claimable by a third party, while the discarded revert reason hides the cause from operators.

RECOMMENDATION

In the factory `_tryInitialize*` helpers, distinguish a facet that is not present from a genuine initializer revert and bubble up the revert reason so a failed initialization aborts the deployment instead of being silently swallowed.

REMEDIATION COMMENT

Addressed in: [b99b6586da0f802356b6ba4a77597fa9936ebb30](#)

SOLVED: The Hashgraph team removed the `Factory::_tryInitialize*` helpers in commit [b99b6586d](#) (PR #1161). `Factory::_initializeSecurityMetadata()` now invokes each initializer directly, including `initializeERC1594()` at `Factory.sol:403` and `initializeERC20Votes()` at `Factory.sol:421`. Any initializer revert now propagates and reverts the complete deployment, so the factory cannot return a partially initialized proxy.

Facet-version lifecycle and key-set invariants are not enforced

• Informational · 0.8

AO:A/AC:L/AX:M/R:P/S:U/C:N/I:L/A:N/D:N/Y:N

DESCRIPTION

`BusinessLogicResolverWrapper::_registerBusinessLogics()` only writes `VersionStatus.ACTIVATED` to `statusByFacetIdAndVersion`. No function on `IBusinessLogicResolver` or `BusinessLogicResolver` exposes a setter for `VersionStatus.DEACTIVATED`, so the state is unreachable at runtime. Independently, `DiamondCutManagerWrapper::createConfiguration()`, `createBatchConfiguration()`, and the internal `_addFacetsToBatchConfiguration()` path never read `statusByFacetIdAndVersion`. Even after a deactivation setter is added, a new configuration would remain composable against a deactivated version unless the facet-addition path also checks the status. The same registration path increments `latestVersionByFacetId` only for caller-supplied entries and never compares the submitted keys with `activeBusinessLogics`, so omitting an active key leaves it at an older latest-version pointer while other keys advance.

SOL 144-155

```

144 businessLogicResolverDataStorage.businessLogics[_businessLogicsRegistryData.businessLogicKey
    ].push(
145     IBusinessLogicResolver.BusinessLogicVersion({
        versionData: IBusinessLogicResolver.VersionData({
146         version: newVersion,
            status: IBusinessLogicResolver.VersionStatus.ACTIVATED
147     })),
        businessLogicAddress: _businessLogicsRegistryData.businessLogicAddress
148     })
149 businessLogicResolverDataStorage.statusByFacetIdAndVersion[
    keccak256(abi.encodePacked(_businessLogicsRegistryData.businessLogicKey, newVersion))
150 ] = IBusinessLogicResolver.VersionStatus.ACTIVATED;
```

`BusinessLogicResolver::getVersionStatus()` exposes the stored value for inspection, but no state-changing interface can set `DEACTIVATED`. Configuration creation resolves the facet address by key and version at `DiamondCutManagerWrapper.sol:174-213` without consulting the stored status. Registration also has no check that every key in `activeBusinessLogics` appears in the submitted array, despite the shared-version behavior documented on `IBusinessLogicResolver`.

Scenario:

1. An operator prepares a registration call to advance the active facet set but omits one previously active key.
2. The submitted keys advance while the omitted key retains its older latest-version pointer.

3. The operator later discovers a defect in a registered facet version and tries to retire it.
4. No on-chain function can move the version to **DEACTIVATED**, and configuration creation does not reject the stored status.
5. The older or retired facet version remains eligible for composition into a future configuration.

Impact: The lifecycle state gives a false expectation of code-enforced version retirement, and a partial registration can produce a facet-version set that differs from the operator's intended bundle. No new privilege is granted, but correct configuration depends on operator discipline and off-chain procedures.

RECOMMENDATION

Enforce the facet-set lifecycle invariants. Add an authorised **DEACTIVATED** setter, reject deactivated versions during configuration creation, and require each registration to include every active key unless an explicit removal operation handles omissions.

REMEDIATION COMMENT

Addressed in: [61435bb513dae3dd9a720866685a142c5efe15be](#)

SOLVED: An administrator-gated version-status setter and deactivated-version checks for new configurations were added, and per-key registration semantics were documented by the **Hashgraph** team. The setter was verified in **BusinessLogicResolver.sol:95-104**, the configuration guard was verified in **DiamondCutManagerWrapper.sol:174-197**, and the final semantics were verified in **IBusinessLogicResolver.sol:8-21** and **IBusinessLogicResolver.sol:114-122**. A version staged before later deactivation can still be activated, a key is not removed by deactivation, and some broader documentation remains inconsistent; these are accepted operational residuals.

Amortization hold release under-subtracts the held balance after an adjustment

• Informational · 0.7 AO:S/AC:L/AX:M/R:P/S:C/C:N/A:N/I:H/D:N/Y:M

DESCRIPTION

`AmortizationStorageWrapper::releaseAmortizationHold()` reads `holdAmount` from the stored hold and passes it to `AmortizationStorageWrapper::_releaseHold()`, which decrements `totalHeldAmountByAccount` by that raw amount. When a balance adjustment (ABAF) has advanced since the hold was created, the stored hold amount is at the old factor while the running total is at the new factor. The release path never calls the balance-adjustment sync (`adjustHoldBalances`) before subtracting, so the totals diverge.

SOL 234-240

```

234      uint256 holdAmount = HoldStorageWrapper.getHold(id_).hold.amount;
      _releaseHold(_tokenHolder, holdInfo.holdId, holdAmount);

235      uint256 releasedHoldId = holdInfo.holdId;
      holdInfo.holdActive = false;

236      s.activeHoldHolders[corporateActionId].remove(_tokenHolder);
237      s.totalHoldByAmortizationId[corporateActionId] -= holdAmount;
```

Scenario:

1. A balance adjustment is scheduled for a future execution date.
2. `setAmortizationHold` creates a hold stamping the current factor.
3. The scheduled adjustment fires; the global ABAF advances and a later hold-touching operation lazily scales `totalHeldAmountByAccount` to the new factor.
4. The amortization hold is released: `_releaseHold` subtracts the old-factor amount from the new-factor total, leaving the account over-credited by the factor delta.

Impact: The released-hold accounting under-subtracts, so `totalHeldAmountByAccount` and `totalHoldByAmortizationId` are left overstated by the factor delta. The discrepancy accumulates across re-hold cycles and skews the amortization hold bookkeeping that gates further amortization operations. It is confined to that internal held-balance accounting and does not reach the ERC20Votes checkpoints or externally consumed voting power.

RECOMMENDATION

Run the balance-adjustment sync (`adjustHoldBalances` or the canonical before-release hook) on the holder before `_releaseHold()` subtracts, so the released amount and the running total are measured at the same adjustment factor.

REMEDIATION COMMENT

Addressed in: [🔗 ae11f9fda9c0fb3ef4a9a55c6b2f9fb6dafc230f](#)

SOLVED: The release path was routed through the canonical hold sync, and the amortization aggregate was given its own adjustment-factor anchor by the [Hashgraph](#) team.

[HoldStorageWrapper.beforeReleaseHold\(\)](#) now runs before the released amount is read at the current factor and subtracted from the resynced aggregate; this was verified in [AmortizationStorageWrapper.sol:662-672](#), with the aggregate rebase in [AmortizationStorageWrapper.sol:707-715](#) and the new LABAF field appended to storage in [AdjustBalancesStorageWrapper.sol:57](#). The duplicate transfer events were removed, and release now emits once through [HoldStorageWrapper.removeAndTransferHoldBalance\(\)](#) at [HoldStorageWrapper.sol:397-406](#). Upgrade handling for holds that predate the fix is provided by [Amortization.migrateAmortizationHoldAccounting\(\)](#), gated by [DEFAULT_ADMIN_ROLE](#), one-shot, and bounded against the current total supply at [Amortization.sol:162-178](#) and [AmortizationStorageWrapper.sol:575-614](#), with the read-only [isAmortizationHoldAccountingMigrationPending\(\)](#) at [Amortization.sol:324-334](#); the [holdOps](#) dependency was added to the facet table in [orchestratorLibraries.ts:165](#). Released and running totals are now measured at the same factor, and the regression coverage was verified in [amortization.test.ts:939-977](#) and [amortization.test.ts:1476-1539](#).

Holder cannot reclaim cleared tokens after their KYC is revoked

• Informational · 0.7 AO:S/AC:L/AX:M/R:N/S:U/C:N/I:N/A:N/D:M/Y:N

DESCRIPTION

`ClearingByPartition::reclaimClearingOperationByPartition()` carries the `onlyIdentifiedAddresses(_clearingOperationIdentifier.tokenHolder, address(0))` modifier, so reclaiming an expired clearing operation requires the holder to still hold valid KYC and a verified identity-registry entry. If the holder's KYC is revoked or their identity entry is removed after the clearing operation locked their tokens, the reclaim path reverts. Cancellation is restricted to `ROLE_CLEARING_VALIDATOR`, so the holder has no other route to recover the locked tokens.

SOL 82-94

```
82 function reclaimClearingOperationByPartition(  
    IClearingByPartition.ClearingOperationIdentifier calldata _clearingOperationIdentifier  
83 )  
    external  
    override  
84    onlyActivated  
85    onlyUnpaused  
86    onlyDefaultPartitionWithSinglePartition(_clearingOperationIdentifier.partition)  
87    onlyWithValidClearingId(_clearingOperationIdentifier)  
88    onlyValidExpirationTimestampForClearing(_clearingOperationIdentifier, true)  
89    onlyIdentifiedAddresses(_clearingOperationIdentifier.tokenHolder, address(0))  
    returns (bool success_)  
90 {
```

Scenario:

1. A clearing operation locks a holder's tokens.
2. Before expiry, the holder's KYC is revoked or their identity-registry entry is removed.
3. After expiry the holder calls `reclaimClearingOperationByPartition()`, which reverts on `onlyIdentifiedAddresses`.
4. Only `ROLE_CLEARING_VALIDATOR` can cancel, so without its action the tokens stay locked.

Impact: A holder whose KYC lapses after a clearing operation is created cannot reclaim their tokens even after expiry, leaving the funds locked with no holder-side recovery path.

RECOMMENDATION

Exempt `ClearingByPartition::reclaimClearingOperationByPartition()` from `onlyIdentifiedAddresses` so a holder can always unwind an expired clearing operation, and

document it as a recovery carve-out since the reclaim returns the holder's own tokens rather than performing a regulated transfer.

REMEDIATION COMMENT

Addressed in: [🔗 16c7d55322ef1fbf045e2d3c8dc04df7eef2387c](#)

SOLVED: The holder identity and KYC gate was removed from expired clearing reclaim by the [Hashgraph](#) team. The external reclaim path was verified in [ClearingByPartition.sol:94-118](#), and all lifecycle branches were verified in [ClearingLifecycleOps.sol:113-307](#). Existing activation, pause, partition, clearing identifier, and expiry checks remain, and the balance is returned to the same holder by each branch.

Clearing hold approved after its expiry is created immediately reclaimable

• Informational · 0.7 AO:S/AC:L/AX:M/R:N/S:U/C:N/I:N/A:M/D:N/Y:N

DESCRIPTION

`ClearingLifecycleOps::clearingHoldCreationExecution()` builds the hold from the `holdExpirationTimestamp` stored at clearing-creation time and passes it to `HoldStorageWrapper::createHoldByPartition()`, which checks only `checkNonZeroHoldAmount()` and never revalidates the expiration. The future-timestamp guard lives only on the external-facing facet modifier used at creation. If the stored `holdExpirationTimestamp` has already passed by the time `ROLE_CLEARING_VALIDATOR` approves, the approval succeeds but the resulting hold is already expired, since `isHoldExpired()` uses `>=`. Reclaiming an expired hold has no identity check.

SOL 273-287

```
273 IHoldTypes.Hold memory hold = IHoldTypes.Hold({
274     amount: holdData.amount,
275     expirationTimestamp: holdData.holdExpirationTimestamp,
276     escrow: holdData.holdEscrow,
277     to: holdData.holdTo,
278     data: holdData.holdData
279 });
280 (bool success, uint256 holdId) = HoldOps.createHoldByPartition(
281     _id.partition,
282     _id.tokenHolder,
283     hold,
284     holdData.operatorData,
285     holdData.operatorType
286 );
```

Scenario:

1. A clearing hold-creation operation is created with a near-term `holdExpirationTimestamp`.
2. The validator approves after that timestamp has passed.
`clearingHoldCreationExecution()` creates the hold with the stale expiry, and `createHoldByPartition()` does not revalidate it.
3. `isHoldExpired()` returns true immediately because the comparison is `>=`, so the hold is born expired.
4. Any caller reclaims the hold at once, returning the tokens to the holder and defeating the collateral lock-up the hold was meant to enforce.

Impact: A clearing hold approved after its stored expiry is created already expired and immediately reclaimable, so the intended collateral lock-up never takes effect.

RECOMMENDATION

In `ClearingLifecycleOps::clearingHoldCreationExecution()`, revert the approval when the stored `holdExpirationTimestamp` is not strictly in the future, so a clearing hold cannot be created already expired.

REMEDIATION COMMENT

Addressed in: [🔗 90847dae60c67127ae76a5c16b51fa6ffdf569b2](#)

SOLVED: Approval-time expiration validation was added by the `Hashgraph` team. Revalidation of the stored expiration and rejection of equal or past timestamps were verified in the common clearing-hold approval path at `ClearingLifecycleOps.sol:263-307`. An already expired and immediately reclaimable hold can no longer be created by a delayed approval.

Operational-status readiness gate is never enforced and any caller can set it

• Informational · 0.6 AO:A/AC:L/AX:L/R:F/S:U/C:N/I:L/A:N/D:N/Y:N

DESCRIPTION

`Initializer::setOperationalStatus()` is a public, unguarded entry point, and the operational-status readiness gate it drives is not enforced anywhere in production. The `onlyOperational` modifier is applied to no business facet, and the factory never calls `initializeInitializer()` or `setOperationalStatus()` during deployment. Because `maxInitializerFacetIndex` stays `0` after a factory deploy, the readiness loop in

`InitializerStorageWrapper::setOperationalStatus()` runs zero iterations, `lastFacetIndex_` stays `0`, and the stored value `lastFacetIndex_ + 1` collides with `1`, the documented fully-operational sentinel, so the configuration is marked operational without a single facet being validated.

SOL 136-139

```
136         isOperational_ = lastFacetIndex_ == facetsLength;
137         unchecked {
138             initializerStorage().configVersionStatus[configId_][versionId_] = isOperational_
139             ? 1 : lastFacetIndex_ + 1;
140         }
```

Scenario:

1. A token proxy is deployed by the factory, which initialises every business facet atomically but never calls `initializeInitializer()`, so `maxInitializerFacetIndex` stays `0`.
2. Any unprivileged account calls the public `Initializer::setOperationalStatus()` once.
3. The readiness loop runs zero iterations, `lastFacetIndex_` stays `0`, and the stored value `lastFacetIndex_ + 1` equals `1`, the fully-operational sentinel.
4. `checkOperational()` now passes for the configuration, although no facet readiness was ever confirmed.

Impact: The multi-step readiness gate the initializer subsystem is meant to provide is non-functional. No production transfer, mint, redeem, coupon, dividend, or administrative path consults operational status, so flipping the sentinel grants no access and blocks nothing today, and the realised security consequence is limited. The risk is latent: if any facet later adopts `onlyOperational` as a real guard, an unprivileged caller could satisfy it without any facet being validated, leaving the control the design relies on bypassable.

RECOMMENDATION

Decide whether the operational-status gate is a real control or dead code. If it is intended to protect live tokens, restrict `Initializer::setOperationalStatus()` to `DEFAULT_ADMIN_ROLE`, have the factory call `initializeInitializer()` during deployment so `maxInitializerFacetIndex` is non-zero, change the partial-progress encoding so `lastFacetIndex_ + 1` cannot collide with the fully-operational sentinel, and apply `onlyOperational` to the business entry points that depend on initialised facets. Otherwise, remove the gate so a future facet does not come to rely on a control that does not work.

REMEDIATION COMMENT

Addressed in: [6701c1fb69d37beda4b40b190ae3fbfd7ee2ba89](#)

SOLVED: The Hashgraph team restricted `Initializer::setOperationalStatus()` to `DEFAULT_ADMIN_ROLE` at `Initializer.sol:41-45` in commit [6701c1fb6](#). Commit [b99b6586d](#) seeds the initializer batch size at `Factory.sol:619-622` and makes equity and bond deployments revert when readiness returns false at `Factory.sol:258-260` and `Factory.sol:289-291`. The remaining index-zero progress encoding is reachable only through a trusted, non-standard administration flow and the supported factory transaction rolls the write back, so it does not preserve the reported unprivileged exploit.

Coupons can be created with backdated record and fixing dates

• Informational · 0.6 AO:S/AC:L/AX:L/R:P/S:U/C:N/I:M/A:L/D:N/Y:N

DESCRIPTION

`Coupon::setCoupon()` guards `recordDate` and `fixingDate` with `onlyValidTimestamp`, which delegates to `DatesValidation::checkTimestamp()`. That function reverts only when the timestamp is zero; it does not compare against `block.timestamp`. A record date in the past is therefore accepted, pinning coupon entitlement to a balance snapshot that has already been finalised.

SOL 44-47

44	<code>onlyValidDates(_newCoupon.recordDate, _newCoupon.executionDate)</code> <code>onlyValidDates(_newCoupon.fixingDate, _newCoupon.executionDate)</code> <code>onlyValidTimestamp(_newCoupon.recordDate)</code>
45	<code>onlyValidTimestamp(_newCoupon.fixingDate)</code>

Scenario:

1. A holder of `ROLE_CORPORATE_ACTION` calls `setCoupon` with a `recordDate` earlier than the current block time.
2. `checkTimestamp` passes because the value is non-zero.
3. The coupon is created and is irrevocable; its entitlement is computed against the already-finalised balance at that past record date.
4. Token holders cannot adjust positions to influence an entitlement whose record date is already in the past.

Impact: Coupons can be backdated to a chosen historical balance distribution, pinning entitlement to a finalised snapshot that holders had no opportunity to react to. The same weak check applies to voting record dates.

RECOMMENDATION

Require `recordDate` and `fixingDate` to be strictly greater than `block.timestamp` in `setCoupon()` (and the voting record-date path) rather than only non-zero.

REMEDIATION COMMENT

Addressed in: [e532da9fb15db47eb1e7e542ea8291db019d8cb7](#)

SOLVED: The Hashgraph team applied strict-future timestamp validation to coupon record and fixing dates in commit `e532da9fb` (PR #1347). On the current source, `Coupon::setCoupon()`

applies `onlyFutureTimestamp` to both inputs at `Coupon.sol:64-65`, and `DatesValidation::checkFutureTimestamp()` rejects timestamps less than or equal to the effective block timestamp at `DatesValidation.sol:13-15`. The same guard protects the voting record-date path at `Voting.sol:41`, confirming the fix.

Upgrading the proxy version invalidates all outstanding permit signatures

• Informational · 0.5 AO:A/AC:L/AX:H/R:F/S:U/C:N/I:L/A:M/D:N/Y:N

DESCRIPTION

`ERC20PermitStorageWrapper::DOMAIN_SEPARATOR()` derives the EIP-712 domain using `Strings.toString(ResolverProxyStorageWrapper.getResolverProxyVersion())` as the version string. The resolver-proxy version is mutable (it changes on every upgrade via the `DiamondCut` update functions), so the domain separator changes whenever the proxy is upgraded. Every EIP-2612 permit signed against the old version then fails signature recovery.

SOL 81-89

```
81      function DOMAIN_SEPARATOR() internal view returns (bytes32) {
82          return
83              _getDomainHash(
84                  ERC20StorageWrapper.getName(),
85                  Strings.toString(ResolverProxyStorageWrapper.getResolverProxyVersion()),
86                  EvmAccessors.getChainId(),
87                  address(this)
88              );
89      }
```

Scenario:

1. A holder signs a permit for the token at configuration version `v`.
2. An admin upgrades the proxy to version `v+1`.
3. `DOMAIN_SEPARATOR()` now returns a different value because the version string changed.
4. The holder's outstanding permit no longer recovers to a valid signer and the relayer's submission reverts.

Impact: A routine upgrade silently invalidates every outstanding permit signature, breaking pending gasless approvals and any integration that pre-signs permits.

RECOMMENDATION

In `ERC20PermitStorageWrapper::DOMAIN_SEPARATOR()`, bind the EIP-712 version field to a fixed signing-domain constant instead of the mutable resolver-proxy version, so upgrades do not invalidate outstanding permits.

REMEDIATION COMMENT

ACKNOWLEDGED: The fail-closed permit-domain design was retained by the `Hashgraph` team. Derivation of the domain from the active configuration version was verified in

`ERC20PermitStorageWrapper.sol:80-88`, and exposure of the current separator was verified in `EIP712.sol:32-34`. A permit is still invalidated by an upgrade between signing and submission, and a new signature is required; this operational effect is accepted by the team.

Permit and protected operations share one nonce and cross-cancel signatures

• Informational · 0.5 AO:A/AC:L/AX:M/R:F/S:U/C:N/I:L/A:L/D:N/Y:N

DESCRIPTION

Both signing schemes use the single per-account counter in `NonceStorageWrapper`.

`ERC20PermitStorageWrapper::permit()` signs the current nonce value `N` and then increments it. The protected-partition path validates with `EIP712::_isNonceValid()`, which requires `_nonce == _currentNonce + 1`, so it signs `N + 1`. Because both paths share the same stored counter, whichever operation lands first moves the counter and invalidates the other's outstanding signature.

SOL 53-60

```
53      uint256 currentNonce = NonceStorageWrapper.getNonceFor(owner);
54      NonceStorageWrapper.setNonceFor(owner);
55      // solhint-disable-next-line func-name-mixedcase
56      address signer = ECDSA.recover(
57          ECDSA.toTypedDataHash(
58              DOMAIN_SEPARATOR(),
59              keccak256(abi.encode(TYPEHASH_ERC20_PERMIT, owner, spender, value,
58              currentNonce, deadline))
```

Scenario:

1. An owner signs a permit and, separately, a protected transfer, both valid against nonce `N`.
2. The permit is submitted; `setNonceFor` advances the counter to `N + 1`.
3. The protected transfer now fails `_isNonceValid` because it expected to consume the same value.
4. The owner must re-sign the operation whose nonce the other consumed.

Impact: The shared nonce couples two independent signing schemes, so an account that holds a pending permit and a pending protected-operation signature has one silently invalidate the other and must re-sign. Hedera orders transactions by fair consensus timestamp with no public mempool, so this is a signature-management correctness issue rather than an adversarial front-running vector.

RECOMMENDATION

Give `ERC20PermitStorageWrapper::permit()` and the protected-partition signed operations independent nonce counters so consuming one does not invalidate the other's pending signature.

REMEDIATION COMMENT

Addressed in: [b9aec4f7a2162ad12b00be049dff5f3e92523825](#)

SOLVED: The nonce domains for permit and each protected operation were separated by the [Hashgraph](#) team. The dedicated direct-hold key was verified in [IProtectedHoldByPartition.sol:10-12](#), its use was verified in [HoldStorageWrapper.sol:116-140](#), the separate clearing-hold key was verified in [ClearingProtectedOps.sol:143-173](#), and the operation-specific getters were verified in [Nonces.sol:45-78](#). Another operation's signature can no longer be invalidated by consuming one signed operation.

Lock can be created already expired and released in the same block

• Informational · 0.5 AO:S/AC:L/AX:L/R:P/S:U/C:N/I:M/A:N/D:N/Y:N

DESCRIPTION

`LockStorageWrapper::requireValidExpirationTimestamp()` validates a new lock with a strict `<`, so `expirationTimestamp == block.timestamp` is accepted.

`LockStorageWrapper::isLockedExpirationTimestamp()` uses `<=`, treating that same value as already expired. The two boundary conditions disagree, so a lock created with expiry equal to the current block time is considered valid at creation yet immediately releasable in the same block.

SOL 295-307

```
295 function isLockedExpirationTimestamp(  
296     bytes32 partition,  
297     address tokenHolder,  
298     uint256 lockId  
299 ) internal view returns (bool) {  
300     return  
301         getLock(partition, tokenHolder, lockId).expirationTimestamp <=  
TimeTravelStorageWrapper.getBlockTimestamp();  
}  
  
302 function requireValidExpirationTimestamp(uint256 expirationTimestamp) internal view {  
303     if (expirationTimestamp < TimeTravelStorageWrapper.getBlockTimestamp())  
304         revert ICommonErrors.WrongExpirationTimestamp();  
}
```

Scenario:

1. A locker creates a lock with `expirationTimestamp` equal to `block.timestamp`.
`requireValidExpirationTimestamp()` passes because the check is strict less-than.
2. Within the same block, `isLockedExpirationTimestamp()` returns true (expired) because its comparison is less-than-or-equal.
3. The release path sees the lock as expired and allows it to be released immediately, so the lock never actually holds the tokens.

Impact: A lock created at the current block time is immediately releasable, defeating the temporal hold. The creation and expiry checks use inconsistent boundaries, so the intended lock window can be zero.

RECOMMENDATION

Align the boundaries by rejecting `expirationTimestamp <= block.timestamp` in `requireValidExpirationTimestamp()` so a newly created lock cannot already be expired.

REMEDIATION COMMENT

Addressed in: [🔗 ea626f3dd2c7cbfcd40f42e98b36ebadc2098174](#)

SOLVED: The shared expiration check was tightened by the [Hashgraph](#) team. Rejection of an expiration timestamp equal to or earlier than the active timestamp was verified in [LockStorageWrapper.sol:345-355](#). Locks, holds, and clearing operations can no longer be created already expired in the same block.

Proxy upgrade entrypoints lack target validation and emit no events

• Informational · 0.5 AO:S/AC:L/AX:L/R:P/S:U/C:N/I:M/A:N/D:N/Y:N

DESCRIPTION

The three proxy upgrade entrypoints in `DiamondCut` mutate the most security-relevant proxy state, the resolver, configuration id and version, with only a registration check and no event.

`updateResolver()` writes the caller-supplied `_newResolver` after a single self-dispatched `checkResolverProxyConfigurationRegistered()`, which any contract that implements the selector and returns can satisfy; there is no zero-address guard and no check that the target implements `IBusinessLogicResolver`. `updateConfig()` and `updateConfigVersion()` gate only on the `(configurationId, version)` pair being registered, so they permit a non-monotonic downgrade to an older version and, for `updateConfig()`, a switch to an unrelated `configurationId` whose facets read the existing storage under different assumptions. None of the three emits an event, so an upgrade is invisible to off-chain monitoring.

SOL 33-43

```
33 function updateResolver(  
34     IBusinessLogicResolver _newResolver,  
35     bytes32 _newConfigurationId,  
36     uint256 _newVersion  
37 ) external override onlyRole(DEFAULT_ADMIN_ROLE) {  
38     _newResolver.checkResolverProxyConfigurationRegistered(_newConfigurationId,  
39     _newVersion);  
    ResolverProxyStorage storage ds = ResolverProxyStorageWrapper.resolverProxyStorage();  
    _updateResolver(ds, _newResolver);  
    _updateConfigId(ds, _newConfigurationId);  
    _updateVersion(ds, _newVersion);  
}
```

Scenario:

1. The `DEFAULT_ADMIN_ROLE` key is compromised or an address is mistyped.
2. A malicious or wrong resolver whose `checkResolverProxyConfigurationRegistered()` returns silently is stored by `updateResolver()`, so every subsequent proxy call resolves its facet through attacker code or bricks.
3. Alternatively `updateConfigVersion()` rolls the proxy back to an older registered version, dropping a control added later, or `updateConfig()` repoints it to an incompatible configuration layout.
4. In every case no event is emitted, so monitoring and indexers see no signal and detection requires polling and diffing `getConfigInfo()`.

Impact: A mistyped or incompatible resolver or config target points the proxy's call routing at the wrong implementation or silently downgrades it to an older facet set, and with no zero-address, interface, or monotonicity check and no emitted event the change can brick the proxy or roll back a security fix invisibly. Upgrade authority is a trusted multisig and timelock by design, so this is a defense-in-depth and observability gap rather than a new unprivileged capability, but once the routing points away from a working resolver the misconfiguration is irreversible from within the proxy.

RECOMMENDATION

In `DiamondCut::updateResolver()` reject the zero address and require the target to implement `IBusinessLogicResolver` via `supportsInterface()`; in `updateConfig()` reject a different `configurationId` unless a migration reconciles the storage layout; and emit a dedicated event from all three entrypoints recording the caller and the old and new resolver, configuration and version. Upgrade authority remains externally enforced by multisig and timelock, so on-chain version monotonicity is not mandated.

REMEDATION COMMENT

Addressed in: [🔗 989442bb2b53f90e5e29a7ce2e13f478fa78dcd2](#)

SOLVED: Resolver-target validation and complete update events were added by the `Hashgraph` team. The resolver checks and four mutators were verified in `DiamondCut.sol:33-138`, the event definitions were verified in `IDiamondCut.sol:12-66`, and the complete old/new event payloads were verified in `DiamondCut.sol:161-192`. Invalid resolver targets are rejected, and every configuration mutation is observable with its prior and new values.

Proceed-recipient init reverts or drops data on array length mismatch

• Informational · 0.5 AO:S/AC:L/AX:L/R:P/S:U/C:N/I:M/A:N/D:N/Y:N

DESCRIPTION

`ProceedRecipientsStorageWrapper::initialize_ProceedRecipients()` iterates over `_proceedRecipients` and indexes `_data[index]` in the same loop, without checking that the two calldata arrays have equal length. The NatSpec states the caller must ensure they match, but nothing enforces it. When `_data` is shorter than `_proceedRecipients`, the checked calldata access reverts with an out-of-bounds panic, failing the deployment. When it is longer, the trailing entries are silently dropped.

SOL 48-60

```
48 function initialize_ProceedRecipients(address[] calldata _proceedRecipients, bytes[]  
   calldata _data) internal {  
   uint256 length = _proceedRecipients.length;  
   for (uint256 index; index < length; ) {  
49       ExternalListManagementStorageWrapper.checkValidAddress(_proceedRecipients[index]);  
       ExternalListManagementStorageWrapper.addExternalList(  
50         STORAGE_LOCATION_PROCEED_RECIPIENTS,  
51         _proceedRecipients[index]  
       );  
       setProceedRecipientData(_proceedRecipients[index], _data[index]);  
52       unchecked {  
         ++index;  
       }  
53   }
```

Scenario:

1. A deployment passes `_proceedRecipients` of length 3 and `_data` of length 2.
2. The loop reaches `index == 2` and reads `_data[2]`, which is out of bounds, reverting with a panic and aborting initialization.
3. If instead `_data` is longer than the recipient array, the surplus entries never execute and are silently lost.

Impact: A length mismatch either reverts the initialization (deployment denial of service) or silently discards data, so the proceed-recipient set is configured incompletely with no error.

RECOMMENDATION

Validate `_proceedRecipients.length == _data.length` at the start of `initialize_ProceedRecipients()` and revert with an explicit error on mismatch.

REMEDIATION COMMENT

Addressed in: [🔗 e450e12ccdfb763e7956ac5e39e6e40308743f89](#)

SOLVED: An equal-length guard was added to the external proceed-recipient initializer by the [Hashgraph](#) team. The guard was verified in [ProceedRecipients.sol:20-32](#), the only storage-wrapper call was verified in [ProceedRecipientsStorageWrapper.sol:54-63](#), and both factory deployment paths were verified in [Factory.sol:242-260](#) and [Factory.sol:318-323](#). Mismatched arrays are now rejected explicitly before indexing or storage updates.

Reverting loan contract permanently blocks loan-portfolio operations

• Informational · 0.4 AO:A/AC:L/AX:H/R:F/S:U/C:N/I:N/A:M/D:N/Y:N

DESCRIPTION

`LoansPortfolioStorageWrapper::notifyLoanHoldingsAssetUpdate()` and `getHoldingsAssetBalances()` call `ILoan(_loanAddress).getLoanDetails()` and `IBalanceTrackerByPartition(...).balanceOfByPartition()` directly, without wrapping them in `try/catch`. A loan or holding asset that reverts or returns malformed data therefore propagates its failure through reclassification or the paginated balance read.

SOL 141-143

141	<code>checkHoldingAssetExists(_holdingsAssetAddress);</code> <code>LoansPortfolioDataStorage storage loanPortfolioStorage = _loansPortfolioStorage();</code> <code>ILoan.LoanDetailsData memory loanDetails =</code>
142	<code>ILoan(_holdingsAssetAddress).getLoanDetails();</code>

Scenario:

1. A loan asset is added to the portfolio.
2. The loan contract later starts reverting on `getLoanDetails()`, or the asset reverts on `balanceOfByPartition()`.
3. Reclassification of that loan reverts, and any paginated balance read containing the asset also reverts.
4. An authorised portfolio manager can still remove the asset because the removal path performs only internal set updates and does not query the loan.

Impact: A single misbehaving loan can block its own reclassification and can make a paginated balance read unavailable for every page containing it, which can leave loan-derived classifications stale until the asset is removed. The asset does not block its own eviction.

RECOMMENDATION

Wrap the `ILoan.getLoanDetails()` and `IBalanceTrackerByPartition.balanceOfByPartition()` calls in `LoansPortfolioStorageWrapper` in `try/catch`. For balance aggregation, expose a per-asset failure result instead of reverting the full page. For reclassification, preserve the existing classification on failure and return an explicit failure result, or provide a trusted manual reclassification path. Keep `removeHoldingsAsset()` independent of loan callbacks so an authorised manager can always evict the offending asset.

REMEDIATION COMMENT

Addressed in: [4bda1f8a90b9c93d8a912a84001625f6f62ccd07](#)

SOLVED: External loan-call failures were isolated by the [Hashgraph](#) team. Failed reclassification reads were verified to preserve the existing classification and return false in [LoansPortfolioStorageWrapper.sol:145-167](#), and per-asset balance failures were verified to return zero without reverting the remaining page in [LoansPortfolioStorageWrapper.sol:353-371](#). The removal path remains independent of external callbacks in [LoansPortfolioStorageWrapper.sol:119-130, 436-459](#).

Reused batch version returns wrong facet data after configuration cancel

• Informational · 0.4 AO:S/AC:L/AX:M/R:P/S:C/C:N/I:M/A:N/D:N/Y:N

DESCRIPTION

`DiamondCutManagerWrapper::_cancelBatchConfiguration()` deletes `addr`, `selectors`, `interfaceIds`, `facetVersions`, `facetIds` and `batchVersion` for a pending batch, but never deletes the `facetIdPosition` entries written by `_addFacetsToBatchConfiguration()`. Because `_startBatchConfiguration()` sets `batchVersion = latestVersion + 1` and cancellation leaves `latestVersion` untouched, the next batch for the same `configurationId` reuses the identical version number. The stale `facetIdPosition[keccak256(configurationId, version, facetId)]` entries from the cancelled batch then collide with the new batch under the same key.

SOL 139-160

```

139 function _cancelBatchConfiguration(bytes32 _configurationId) internal returns (uint256
    batchVersion_) {
    DiamondCutManagerStorage storage dcms = _diamondCutManagerStorage();
140     batchVersion_ = _getBatchConfigurationVersion(_configurationId);
    bytes32 configVersionHash = _buildHash(_configurationId, batchVersion_);
141
    bytes32[] storage facetIds = dcms.facetIds[configVersionHash];
    uint256 facetIdsLength = facetIds.length;
142
    for (uint256 index; index < facetIdsLength; ) {
        bytes32 configVersionFacetHash = _buildHash(_configurationId, batchVersion_,
143         facetIds[index]);
        delete dcms.addr[configVersionFacetHash];
        _cleanSelectors(dcms, _configurationId, batchVersion_, configVersionFacetHash);
144         _cleanInterfacesIds(dcms, _configurationId, batchVersion_, configVersionFacetHash);
        unchecked {
145             ++index;
146         }
147     }
148
    delete dcms.facetVersions[configVersionHash];
    delete dcms.facetIds[configVersionHash];
149     delete dcms.batchVersion[_configurationId];
150 }

```

Scenario:

1. An operator creates a configuration with three facets A, B, C at version 2, then calls `cancelBatchConfiguration()`. The loop clears the facet data but leaves `facetIdPosition` for A=1, B=2, C=3 in storage.
2. The operator starts a revised batch for the same configuration. Since `latestVersion` is still 1, the new batch reuses version 2 and registers only one facet D with `facetIdPosition[D]=1` and a single-element `facetVersions` array.

3. The batch is activated and `latestVersion` becomes 2.
4. `getFacetVersionByConfigurationIdVersionAndFacetId(configurationId, 2, A)` finds the stale position 1, passes the `pos == 0` guard, and returns facet D's version instead of reverting with `FacetIdNotRegistered`.
5. The same call for B reads `facetVersions[...][1]` against a one-element array and reverts with an out-of-bounds panic.

Impact: Facet-version reads for the diamond return wrong data or revert after a batch is cancelled and recreated. `InitializerStorageWrapper::currentFacetVersion()` and `setOperationalStatus()` consume these lookups, so a reused configuration version can mark the wrong facets ready or fail version resolution for the affected facets.

RECOMMENDATION

Delete the position entry inside the cancel loop, adding `delete` `dcms.facetIdPosition[_buildHash(_configurationId, batchVersion_, facetIds[index])]` alongside the existing `addr`, selector and interface cleanup in `_cancelBatchConfiguration()`.

REMEDIATION COMMENT

Addressed in: [🔗 1e8b3cb3707ccb2fe7a7c8d1fbe6d25ad7c7c065](#)

SOLVED: Every cancelled facet-position entry was removed by the `Hashgraph` team. Use of the same configuration, version, and facet hash by the write, cleanup, and lookup paths was verified in `DiamondCutManagerWrapper.sol:175-215`, `DiamondCutManagerWrapper.sol:225-244`, and `DiamondCutManagerWrapper.sol:559-570`. Stale facet data can no longer be returned when a cancelled version is reused.

Factory TREX deploy leaves compliance unbound and can revert the whole suite

• Informational · 0.3 AO:S/AC:L/AX:M/R:F/S:C/C:N/I:M/A:M/D:N/Y:N

DESCRIPTION

`TREXBaseDeploymentLib::deployTREXSuite()` takes the else-branch when a non-zero `_compliance` is supplied. The deploy-new branch calls `_token.setCompliance()` and `mc.bindToken()`, but the else-branch only assigns `mc = IModularCompliance(_compliance)` and performs neither binding.

SOL 51-60

```

51     IModularCompliance mc;
52     uint256 transferOwnership; // Bit 0 tracks MC and bit 1 IR
53     if (_compliance == address(0)) {
54         mc = IModularCompliance(_deployMC(_salt, _implementationAuthority));
55         _token.setCompliance(address(mc));
56         mc.bindToken(address(_token));
57         transferOwnership = 1;
58     } else {
59         mc = IModularCompliance(_compliance);
60     }

```

The code then still calls `mc.addModule()` / `mc.callModuleFunction()` and transfers MC ownership to the owner. A T-Rex `ModularCompliance` only services `transferred` / `created` / `destroyed` / `canTransfer` for the single token it is bound to, so an externally supplied MC that was never `bindToken`-ed to this token rejects the binding-dependent calls or evaluates rules against the wrong token, and the factory never asserts `_token.compliance() == _compliance`. Separately, the suite finalises with `token.renounceRole(ROLE_TREX_OWNER)` then `token.renounceRole(DEFAULT_ADMIN_ROLE)`; `renounceRole` is `onlyActivated` `onlyUnpaused` and runs `checkNotSoleAdmin`, so it hard-reverts if the token is not yet activated and unpaused or if `_tRexOwner` equals the factory, reverting the whole atomic deploy.

Scenario:

1. The owner supplies an existing modular-compliance contract at deploy (an intended option).
2. The else-branch skips `setCompliance()` and `bindToken()`, so the token and the supplied MC are not bound to each other.
3. Compliance hooks then evaluate against the wrong bound token or revert, shipping a token whose compliance checks are not authoritative.
4. In a separate configuration where `_tRexOwner` equals the factory or the token is not yet activated, the finalising `renounceRole(DEFAULT_ADMIN_ROLE)` reverts and the entire

CREATE2 suite deploy fails.

Impact: When an existing modular-compliance contract is supplied, the token deploys with both the token-to-compliance and the compliance-to-token bindings missing, so a regulated security can go live without a wired compliance module and its transfers are not subject to the compliance checks the standard expects until an operator notices and rebinds it. Some owner configurations also make the finalising `renounceRole` calls revert the entire CREATE2 suite deploy. The binding gap is admin-recoverable once detected, but until then the regulatory guarantee that every transfer is compliance-evaluated does not hold.

RECOMMENDATION

- In the supplied-compliance branch of `TREXBaseDeploymentLib::deployTREXSuite()`, call `mc.bindToken()` and `_token.setCompliance()` and assert `_token.compliance() == _compliance` before pushing module settings.
- Finalize by granting the owner its roles and dropping only the factory's own membership via `revokeRole(role, address(this))` after the token is activated and unpaused.

REMEDIATION COMMENT

Addressed in: [🔗 96f0781a296bbc2dd83fddf051f2e50769f43c96](#)

SOLVED: Commit [96f0781a296bbc2dd83fddf051f2e50769f43c96](#) removed the affected T-REX deployment libraries and suite-deployment surfaces. Commit [39500f87153b42becbb2763dbaed7638c5479820](#) removed the remaining `factory/ERC3643` subtree and scaffolding. The current generic `ICompliance` adapter has no `bindToken()` or modular-suite deployment semantics, so the reported supplied-compliance binding and T-REX finalization paths no longer exist.

ISIN validator accepts non-alphanumeric characters as valid identifiers

• Informational · 0.3 AO:S/AC:L/AX:L/R:P/S:U/C:N/I:L/A:N/D:N/Y:N

DESCRIPTION

`isinValidator::_byteToCode()` converts each ISIN character to a numeric code without checking that the byte falls within the ISO 6166 character set (digits 48 to 57, uppercase letters 65 to 90). It applies `code_ - _ASCII_7` for any byte above `_ASCII_9` and `code_ - _ASCII_0` otherwise, so bytes such as ASCII 58 to 64 (the `;<=>?@` range) map onto the same code space as valid characters and are indistinguishable in the checksum.

SOL 71-74

```
71 function _byteToCode(bytes1 _character) pure returns (uint8 code_) {
    code_ = uint8(_character);
72     code_ = code_ > _ASCII_9 ? code_ - _ASCII_7 : code_ - _ASCII_0;
73 }
```

Scenario:

1. A token is deployed with an ISIN string containing a non-alphanumeric byte, for example a character in the ASCII 58 to 64 range.
2. `_byteToCode()` maps the byte to a numeric code with the same arithmetic used for valid characters, so the checksum still computes.
3. The on-chain ISIN check passes and the token is created with a malformed identifier.

Impact: Non-standard ISINs containing out-of-range characters pass on-chain validation, so tokens can be created with identifiers that are not valid ISO 6166 codes.

RECOMMENDATION

Reject any byte outside the digit range 48 to 57 and uppercase range 65 to 90 in `_byteToCode()` before converting it, reverting on an invalid ISIN character.

REMEDIATION COMMENT

Addressed in: [5c0caa9688282549e9955877e94b5573b96a3f23](#)

SOLVED: Commit [5c0caa9688282549e9955877e94b5573b96a3f23](#) removed `isinValidator.sol` and all ISIN and country deployment fields. The current factory input structures have no equivalent identifier field or validation path, so the reported checksum issue is removed with the affected functionality.

ERC1594 `isIssuable` flag is never enforced on any mint path

• Informational · 0.3 AO:S/AC:L/AX:L/R:P/S:U/C:N/I:L/A:N/D:N/Y:N

DESCRIPTION

`ERC1594StorageWrapper::issue()` mints directly without consulting `isIssuable()`. The `issuance` flag is set to `true` at initialization and has no setter that can turn it `false`, and none of the issuance write paths (`issue()`, `mint()`, `batchMint()`, `issueByPartition()`) read it. ERC1594 requires `issue()` to revert unless `isIssuable()` returns `true` and the flag to be permanently disableable. Here `isIssuable()` is a read-only view with no effect on minting.

SOL 81-87

```
81 function issue(address tokenHolder, uint256 value) internal {
    ERC20StorageWrapper.mint(tokenHolder, value);
82 }
83 function isIssuable() internal view returns (bool) {
84     return erc1594Storage().issuance;
85 }
```

Scenario:

1. An integrator reads `isIssuable()` expecting the standard guarantee that issuance can be finalised and frozen.
2. No code path exists to set `issuance` to `false`, so the flag stays `true` for the life of the token.
3. Even if it could be set false, `issue()`, `mint()` and `batchMint()` mint regardless because none check the flag.

Impact: The ERC1594 `isIssuable()` guarantee is not honoured. Tokens advertised as ERC1594-compliant cannot finalise issuance, and tooling that gates on `isIssuable()` is misled because the flag never reflects an enforceable state.

RECOMMENDATION

Gate `issue()`, `mint()`, `batchMint()` and `issueByPartition()` on `isIssuable()` and add a one-way setter that permanently clears `issuance`, matching the ERC1594 requirement.

REMEDIATION COMMENT

Addressed in: [🔗 8847126e656eca261927d631b98bb0ff0f2b497b](#)

SOLVED: A permanent issuance-finalization control was added by the `Hashgraph` team. The one-way state transition and non-repeatable initializer were verified in `ERC1594StorageWrapper.sol:67-70,109-129` and `InitializerStorageWrapper.sol:261-269`. Issuance-state enforcement was verified across the single, batch, and partition mint entry points in `Mint.sol:35-74`, `BatchMint.sol:38-70`, and `MintByPartition.sol:36-57`.

Unvalidated KPI bounds cause division-by-zero or underflow, bricking coupons

• Informational · 0.2 AO:S/AC:L/AX:H/R:P/S:U/C:N/I:N/A:M/D:N/Y:N

DESCRIPTION

`KpiLinkedRateLib::_getDecreasedRate()` divides by `kpiData.baseLine - kpiData.maxDeviationFloor` and `_getIncreasedRate()` divides by `kpiData.maxDeviationCap - kpiData.baseLine`. The KPI configuration setter does not enforce `maxDeviationFloor < baseLine < maxDeviationCap`. If `maxDeviationFloor == baseLine` (or `maxDeviationCap == baseLine`) the denominator is zero and the call panics; if the ordering is inverted the subtraction underflows and panics in checked arithmetic.

SOL 151-152

```
151     uint256 impactDeltaRate = (factor * (kpiData.baseLine - impactData)) /  
152         (kpiData.baseLine - kpiData.maxDeviationFloor);
```

Scenario:

1. KPI parameters are configured with `maxDeviationFloor` equal to (or above) `baseLine`.
2. A coupon's impact data falls below `baseLine`, routing rate resolution into `_getDecreasedRate`.
3. The denominator `baseLine - maxDeviationFloor` is zero (or the subtraction underflows), and the call panics.
4. Rate resolution reverts on every attempt, so the coupon can never be finalised.

Impact: Misordered KPI bounds make rate calculation panic permanently, blocking coupon resolution for the affected KPI configuration with no on-chain recovery.

RECOMMENDATION

Validate `maxDeviationFloor < baseLine < maxDeviationCap` when KPI parameters are set, so the rate-calculation denominators can never be zero or underflow.

REMEDIATION COMMENT

Fixed in current source: `requireValidImpactData` enforces strict `maxDeviationFloor < baseLine < maxDeviationCap` on all KPI write paths, eliminating the division-by-zero queue-deadlock.

Scheduled-task dispatch can drop or silently consume a failed task

• Informational · 0.1 AO:S/AC:M/AX:H/R:P/S:U/C:N/I:M/A:N/D:N/Y:N

DESCRIPTION

`ScheduledTasksStorageWrapper::triggerScheduledTasks()` pops a task before dispatching it and wraps the dispatch in a bare `catch {}` that catches every error, including out-of-gas. Under EIP-150 the parent frame keeps only 1/64 of the gas after a sub-call runs out, which can be too little for `_onTaskExecutionFailed()` to run `cancelCorporateAction()` and emit its event, so the already-popped task is lost without a clean cancellation. Independently, `ScheduledTasksDispatchOps::execute()` decodes `task.data` with `abi.decode(task.data, (bytes32))` without a length check, reverting with `Panic(0x22)` on short data, and has no branch for an unknown `callbackType`, returning `bytes32(0)` silently, indistinguishable from a successful zero-return cross-ordered task. In both cases the task was already popped.

SOL 109-117

```
109 ScheduledTasksLib.popScheduledTask(_scheduledTasks);
110 try ScheduledTasksDispatchOps.execute(callbackType, currentScheduledTask) returns (bytes32
111 subTaskType) {
112     if (subTaskType != bytes32(0)) {
113         _triggerOneSubTask(subTaskType, currentBlockTimestamp);
114     }
115 } catch {
116     _onTaskExecutionFailed(callbackType, currentScheduledTask);
117 }
```

Scenario:

1. A task is popped and `execute()` consumes nearly all remaining gas; the bare `catch` fires with only the 1/64 remainder and `_onTaskExecutionFailed()` cannot cancel the corporate action.
2. Or the task carries data shorter than 32 bytes, so `abi.decode()` reverts with an out-of-bounds panic.
3. Or the task has an unrecognised `callbackType`, so `execute()` returns `bytes32(0)` and performs no operation.
4. In each case the popped task is neither executed nor cleanly cancelled.

Impact: A scheduled task can be removed from the queue and then lost, panic the dispatcher, or be silently consumed as a no-op, leaving the associated corporate action unresolved with no record distinguishing the failure modes.

RECOMMENDATION

Reserve and check a minimum gas floor before dispatching each scheduled task so

`_onTaskExecutionFailed()` can always cancel the corporate action and record the failure, and in `ScheduledTasksDispatchOps::execute()` validate that `task.data` is at least 32 bytes and revert on an unrecognised `callbackType` instead of returning `bytes32(0)`. This preserves the intended pop-and-continue queue behaviour while ensuring a failed or malformed task is never dropped or consumed silently.

REMEDIATION COMMENT

Addressed in: [🔗 9879296bb48f533a8c91f45c1c53b42a60fde18f](#)

SOLVED: Commit [9879296bb48f533a8c91f45c1c53b42a60fde18f](#) removed the bare dispatch catch. A dispatch failure now reverts the full transaction and restores all queue pops. Production task constructors always encode a `bytes32`, and callback routing exposes only `crossOrdered` plus the three recognized sub-task types. The unchecked decode and implicit default-return branches are therefore unreachable through the current token call graph. Role-gated force-cancel entry points provide recovery for a persistently failing corporate action.

Operator clearing cancel and reclaim mint an allowance to address(0)

• Informational · AO:S/AC:L/AX:M/R:P/S:U/C:N/A:N/I:N/D:N/Y:N

DESCRIPTION

`ClearingLifecycleOps::restoreClearingAllowance()` runs on the cancel and reclaim paths for both authorised (allowance-based) and operator clearings. It reads the recorded third-party via `ClearingStorageWrapper::getClearingThirdParty()` and calls `TokenCoreOps.increaseAllowedBalance(holder, thirdParty, amount)`. The third-party slot is written only by `ClearingOps::decreaseAllowedBalanceForClearing()`, which runs exclusively on the spender-initiated allowance path; operator clearings never populate the slot. `getClearingThirdParty()` therefore returns the default `address(0)` for operator clearings, and the subsequent `increaseAllowedBalance()` credits an allowance from the holder to the zero address and emits a matching `Approval(holder, 0x0, amount)` event.

SOL 360-376

```

360 function restoreClearingAllowance(
361     IClearingTypes.ClearingOperationIdentifier calldata _id,
362     uint256 _amount
363 ) private {
364     ThirdPartyType operatorType = ClearingStorageWrapper.getClearingThirdPartyType(_id);
365     if (operatorType != ThirdPartyType.AUTHORIZED && operatorType !=
366         ThirdPartyType.OPERATOR) return;
367     TokenCoreOps.increaseAllowedBalance(
368         _id.tokenHolder,
369         ClearingStorageWrapper.getClearingThirdParty(
370             _id.partition,
371             _id.tokenHolder,
372             _id.clearingOperationType,
373             _id.clearingId
374         ),
375         _amount
376     );
377 }

```

Scenario:

1. An operator triggers a clearing (transfer, redeem, or hold-creation) on behalf of a holder via the operator-clearing surface. `ClearingLifecycleOps` stores the clearing with `ThirdPartyType.OPERATOR`. `ClearingStorageWrapper::setClearingThirdParty()` is never called, so the stored third-party slot remains `address(0)`.
2. The clearing is cancelled or reclaimed before approval. `ClearingLifecycleOps::restoreClearingAllowance()` runs, treats `OPERATOR` the same as `AUTHORIZED`, reads the empty third-party slot as `address(0)`, and calls `TokenCoreOps.increaseAllowedBalance(_id.tokenHolder, address(0), _amount)`.

3. `ERC20Storage.allowed[holder][address(0)]` is increased by `_amount`, and an `Approval(holder, address(0), amount)` event is emitted.

Impact: The phantom allowance is not spendable today: `transferFrom` cannot resolve `msg.sender == address(0)`, so no caller can drain it. The allowance record is nevertheless inconsistent with what off-chain indexers and accounting tools observe, and the zero-address entry grows monotonically across cancels and reclaims. If a future change ever populates the operator third-party slot, or an integration reads `allowed[holder][address(0)]` without filtering the sentinel, the accumulated amount materialises as a real over-credit.

RECOMMENDATION

Restrict `ClearingLifecycleOps::restoreClearingAllowance()` to `ThirdPartyType.AUTHORIZED` only, or reject a zero third-party address before calling `TokenCoreOps.increaseAllowedBalance()`. Both fixes remove the phantom credit and neither changes the authorised-allowance restore path.

REMEDIATION COMMENT

Addressed in: [🔗 ccc4716c422da58eda9cc7442bdf7d7ab4fc6d3b](#)

SOLVED: The allowance restore path was restricted to allowance-based clearings, and a zero-address guard was added by the `Hashgraph` team. The early return for non `AUTHORIZED` clearings and the rejection of a zero spender before `TokenCoreOps.increaseAllowedBalance()` were verified in `ClearingLifecycleOps.sol:364-378`, and population of the third-party slot only on the spender-initiated path was verified in `ClearingOps.sol:306-322`. Cancelling or reclaiming an operator clearing no longer credits an allowance to `address(0)` or emits a phantom `Approval` event, and the authorised-allowance restore path is unchanged.

ResolverProxy accepts native HBAR without a rescue path in the current configuration

• Informational · AO:A/AC:L/AX:M/R:F/S:U/C:N/A:N/I:N/D:N/Y:N

DESCRIPTION

`ResolverProxy` declares `receive() external payable {}` and a payable `fallback()`. Native HBAR sent to either path becomes part of `address(this).balance`, and no facet in the current contract tree exposes a withdraw, sweep, or rescue function. However, `DiamondCut::updateConfigVersion()`, `updateConfig()`, and `updateResolver()` let a `DEFAULT_ADMIN_ROLE` holder select a registered configuration that contains a rescue facet. Because the fallback executes the facet with `delegatecall`, that facet runs in the proxy context and can transfer the existing proxy balance. Recovery therefore depends on a live default-admin upgrade path but does not require redeploying the proxy.

SOL 44-47

```
44 |     receive() external payable {}
45 |
46 |     // solhint-disable-next-line no-complex-fallback
47 |     fallback() external payable {
```

ATS tokens have no native-asset business model, so accepting plain HBAR without an immediately available rescue function remains an operational and documentation gap.

Scenario:

1. An ATS proxy is deployed through the factory.
2. An external account sends non-zero `msg.value` to the proxy, by mistake or through the payable fallback.
3. The value remains in `address(this).balance` because the current facet configuration has no rescue selector.
4. A live `DEFAULT_ADMIN_ROLE` holder registers a configuration with an admin-gated rescue facet, updates the proxy through `DiamondCut`, and calls the rescue function in the proxy context.

Impact: Native value is inaccessible under the current configuration, but it is recoverable through the existing admin-controlled configuration upgrade path. The residual concern is operational clarity and continued availability of the default-admin role, not permanent loss caused by an immutable contract limitation.

RECOMMENDATION

Include an admin-gated rescue function in the supported facet configuration, or reject direct native transfers if the payable receive path is not required. Document that recovery through a later configuration requires a live `DEFAULT_ADMIN_ROLE` holder and that the rescue facet executes against the existing proxy balance through `delegatecall`.

REMEDIATION COMMENT

ACKNOWLEDGED: The payable proxy surface was retained without a rescue function by the `Hashgraph` team. The payable constructor, `receive()`, and `fallback()` were verified in `ResolverProxy.sol:32-53`, and the absence of native-balance handling in the facet tree was confirmed. Recovery through the admin-gated `DiamondCut.updateConfigVersion()`, `updateConfig()`, and `updateResolver()` paths was verified in `DiamondCut.sol:53-135`; a rescue facet registered in a later configuration executes in the proxy context against the existing balance. No native-asset business model exists, exposure is limited to incidental amounts, and recovery remains dependent on a live `DEFAULT_ADMIN_ROLE` holder; this operational condition is accepted by the team.

Selector blacklist enforcement is registration-time only and is not documented as such

• Informational · A0:S/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N

DESCRIPTION

`DiamondCutManagerWrapper::_checkSelectorsBlacklist()` is called by `_registerSelectors()` when a configuration is created. The dispatch path that serves proxy calls reads the stored `facetAddress[hash(configId, version, selector)]` mapping and does not consult the blacklist. Adding a selector to `selectorBlacklist[configId]` after a configuration is active therefore has no effect on proxies bound to that configuration; the check only rejects the selector when a later configuration version is composed.

SOL 268-283

```
268 function _resolveResolverProxyCallV2(  
269     bytes32 _configurationId,  
270     uint256 _version,  
271     bool _replacementEnabled,  
272     bytes4 _selector  
273 ) internal view returns (address facetAddress_) {  
    facetAddress_ = _diamondCutManagerStorage().facetAddress[  
274         _buildHashSelector(_configurationId, _version, _selector)  
    ];  
275     if (_replacementEnabled) {  
        address replacementAddress = _getReplacementAddress(facetAddress_);  
        if (replacementAddress != address(0)) {  
276             facetAddress_ = replacementAddress;  
277         }  
278     }  
279 }
```

The implementation NatSpec on `BusinessLogicResolver` now states that the mutators affect future configuration validation. However, `infrastructure/README.md` still describes the blacklist as selectors that should not be delegated and as a function-access control, and the interface NatSpec at `IBusinessLogicResolver.sol:102-114` does not state the registration-time scope.

Scenario:

1. An operator discovers that a selector in an active configuration should no longer be available.
2. The operator adds the selector to the configuration blacklist and expects the documented control to stop live delegation.
3. The existing selector mapping remains unchanged, and `_resolveResolverProxyCallV2()` continues to return the registered facet address.

4. The operator must compose a new configuration version without the selector and migrate each affected proxy to that version.

Impact: No runtime security control is bypassed because the blacklist is implemented as a registration guard. The risk is documentation and interface ambiguity that can cause an operator to use the wrong incident-response procedure.

RECOMMENDATION

Clarify the scope in three places: (1) update `infrastructure/README.md` to state the blacklist rejects selectors at configuration-registration time only, and does not disable already-registered selectors on live proxies; (2) update the NatSpec on `IBusinessLogicResolver.addSelectorsToBlacklist` and `removeSelectorsFromBlacklist` with the same scope; (3) document the operational path for retiring an already-registered selector, which is composing a new configuration version that omits the selector and migrating each proxy to the new version.

REMEDIATION COMMENT

Addressed in: [10f49b8530a0c5e7816caedd10cfc76138c07f20](#)

SOLVED: The registration-time scope of the selector blacklist was documented in the interface, and a regression test pinning the boundary was added by the `Hashgraph` team. The NatSpec on `addSelectorsToBlacklist()` and `removeSelectorsFromBlacklist()` was verified in `IBusinessLogicResolver.sol:124-139` and `BusinessLogicResolver.sol:50-66`, the README wording was verified in `infrastructure/README.md:132`, and the post-activation test was verified in `diamondCutManager.test.ts:681-708`. The mechanism is unchanged: `_registerSelectors()` still enforces the blacklist at `DiamondCutManagerWrapper.sol:754`, and dispatch at `DiamondCutManagerWrapper.sol:272` does not consult it.

Disclaimer

Halborn strongly recommends conducting a follow-up assessment of the project either within six months or immediately following any material changes to the codebase, whichever comes first. This approach is crucial for maintaining the project's integrity and addressing potential vulnerabilities introduced by code modifications.